

АННОТАЦИЯ

Выпускная квалификационная работа посвящена исследованию, проектированию и разработке веб-приложения для совместного учета расходов в группах пользователей при помощи FastAPI, PostgreSQL, RabbitMQ, React и Docker. Работа охватывает анализ предметной области, сравнительный анализ существующих решений, формализацию финансовой модели групповых расчетов и проектирование распределенной архитектуры из четырех независимых сервисов — аутентификации, групп, расходов и уведомлений.

Целью работы является разработка веб-приложения для совместного учета расходов на основе исследования современных архитектурных подходов и принципов проектирования распределенных систем, обеспечивающего финансовую корректность операций и согласованность данных между сервисами.

В рамках работы выполнен сравнительный анализ существующих решений, обоснован выбор технологического стека, спроектированы схема базы данных и REST API, реализованы серверная и клиентская части приложения, организована виртуализация на основе Docker Compose и маршрутизация на Nginx, разработан алгоритм минимизации числа взаимных переводов и проведено тестирование функциональных и нефункциональных требований к системе.

Выпускная квалификационная работа изложена на 106 страницах, содержит 5 рисунков, 10 таблиц, 5 приложений и список использованной литературы из 41 наименования.

ANNOTATION

The graduation qualification work is devoted to the research, design and development of a web application for shared expense accounting in user groups based on the FastAPI, PostgreSQL, RabbitMQ, React and Docker stack. The work covers domain analysis, a comparative review of existing solutions, formalisation of a financial model of group settlements and the design of a distributed architecture composed of four independent services — authentication, groups, expenses and notifications.

The aim of the work is to develop a web application for shared expense accounting based on a study of contemporary architectural approaches and principles of distributed system design, ensuring financial correctness of operations and data consistency across services.

Within the work, a comparative analysis of existing solutions is carried out, the technology stack is justified, the database schema and the REST API are designed, the backend and frontend parts are implemented, the deployment infrastructure based on Docker Compose and Nginx is set up, an algorithm for minimising the number of mutual transfers is developed, and functional and non-functional requirements of the system are tested.

The work is presented on 106 pages, contains 5 figures, 10 tables, 5 appendices and a bibliography of 41 references.

СОДЕРЖАНИЕ

<u>ВВЕДЕНИЕ.....</u>	<u>9</u>
<u>1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И ПРОЕКТИРОВАНИЕ СИСТЕМЫ.....</u>	<u>13</u>
1.1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	13
1.2 ОБЗОР И СРАВНИТЕЛЬНЫЙ АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ.....	19
1.3 АРХИТЕКТУРНЫЙ АНАЛИЗ АНАЛОГОВ И ОБОСНОВАНИЕ ТЕХНОЛОГИЧЕСКОГО СТЕКА.....	23
1.4 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ.....	26
1.5 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ И API.....	32
1.6 ФИНАНСОВАЯ МОДЕЛЬ.....	37
Выводы по разделу 1.....	41
<u>2 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ.....</u>	<u>43</u>
2.1 РЕАЛИЗАЦИЯ СЕРВЕРНОЙ ЧАСТИ ПРИЛОЖЕНИЯ.....	43
2.2 РЕАЛИЗАЦИЯ КЛИЕНТСКОЙ ЧАСТИ ПРИЛОЖЕНИЯ.....	47
2.3 РЕАЛИЗАЦИЯ ИНФРАСТРУКТУРЫ ЗАПУСКА ПРИЛОЖЕНИЯ.....	55
2.4 ТЕСТИРОВАНИЕ И ОЦЕНКА СООТВЕТСТВИЯ ТРЕБОВАНИЯМ.....	58
Выводы по разделу 2.....	66
<u>3 АНАЛИЗ РЕЗУЛЬТАТОВ РАЗРАБОТКИ И НАПРАВЛЕНИЯ РАЗВИТИЯ СИСТЕМЫ</u>	<u>68</u>
3.1 АНАЛИЗ РЕЗУЛЬТАТОВ РЕАЛИЗАЦИИ АРХИТЕКТУРНЫХ РЕШЕНИЙ.....	68
3.2 СОПОСТАВЛЕНИЕ РЕАЛИЗОВАННОЙ СИСТЕМЫ С АЛЬТЕРНАТИВНЫМИ АРХИТЕКТУРНЫМИ ПОДХОДАМИ.....	71
3.3 ОЦЕНКА ЭФФЕКТИВНОСТИ И КАЧЕСТВА РЕАЛИЗОВАННОГО РЕШЕНИЯ.....	75
3.4 НАПРАВЛЕНИЯ РАЗВИТИЯ СИСТЕМЫ.....	79
Выводы по разделу 3.....	82
<u>ЗАКЛЮЧЕНИЕ.....</u>	<u>84</u>
<u>СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....</u>	<u>88</u>
<u>ПРИЛОЖЕНИЕ А – ER-ДИАГРАММЫ БАЗЫ ДАННЫХ.....</u>	<u>93</u>

<u>ПРИЛОЖЕНИЕ Б – ПОЛНЫЙ КАТАЛОГ REST API.....</u>	<u>94</u>
СЕРВИС АУТЕНТИФИКАЦИИ (AUTH-SERVICE).....	94
СЕРВИС ГРУПП (GROUPS-SERVICE).....	95
<u>ПРИЛОЖЕНИЕ В – КОНФИГУРАЦИЯ ЗАПУСКА ПРИЛОЖЕНИЯ.....</u>	<u>98</u>
<u>ПРИЛОЖЕНИЕ Г – МАТРИЦА ТЕСТ-КЕЙСОВ.....</u>	<u>102</u>
Г.1 ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ.....	102
Г.2 НЕФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ.....	103
<u>ПРИЛОЖЕНИЕ Д – АРХИТЕКТУРА СИСТЕМЫ.....</u>	<u>106</u>

ВВЕДЕНИЕ

Актуальность работы состоит в том, что задача автоматизации совместного учета расходов в группах пользователей опирается на устойчивые демографические и поведенческие тенденции. По данным аналитического центра «Дом.РФ» и ВЦИОМ, с 2020 по 2025 год число российских семей, арендующих жилье, выросло с 5,1 до 5,8 млн [1], а исследование НАФИ совместно с программой «Больше, чем путешествие» зафиксировало, что 76 % российских молодых людей в возрасте 14–35 лет выбирают форматы группового туризма со средним размером группы около пяти человек [2]. Эти группы сталкиваются с задачей распределения совместных расходов между участниками.

Решение задачи без специализированных средств — в заметках, чатах, таблицах — неустойчиво при росте числа участников и количества операций. Некоторые коммерческие сервисы (Splitwise [3], Tricount [4]) закрывают эту задачу, однако имеют закрытый исходный код и ограниченную русскоязычную локализацию; полноценных русскоязычных аналогов с открытым исходным кодом сравнительный анализ §1.2 не выявляет.

Инженерная сторона задачи: совместный учет расходов в распределенной системе требует одновременно финансовой корректности операций, согласованности данных между независимыми сервисами и идемпотентной обработки повторных запросов и доставленных событий. Эти требования задают конкретные проектные ограничения и определяют выбор архитектурных решений, проработанных в работе.

Объект исследования — процесс учета и распределения совместных расходов в группах пользователей.

Предмет исследования — методы проектирования и реализации веб-приложения для автоматизации групповых финансовых расчетов.

Цель работы — разработка веб-приложения для совместного учета расходов на основе исследования архитектурных подходов и принципов проектирования распределенных систем.

Для достижения поставленной цели сформулированы следующие задачи.

Провести анализ предметной области, сравнительный анализ существующих решений и обосновать выбор архитектурного подхода и технологического стека.

Спроектировать архитектуру системы, схему базы данных, программный интерфейс и формализовать финансовую модель групповых взаиморасчетов.

Реализовать серверную и клиентскую части приложения, а также инфраструктуру для запуска приложения.

Провести тестирование приложения и оценить соответствие реализации функциональным и нефункциональным требованиям.

Выполнить анализ результатов реализации, сопоставить их с альтернативными архитектурными подходами и определить направления развития системы.

Методы исследования — анализ научной литературы и существующих программных решений, объектно-ориентированное проектирование, анализ вариантов использования для формализации функциональных требований, автоматизированное тестирование, включая тестирование по свойствам финансовых инвариантов.

Научная новизна работы включает два элемента, полученных при решении задач 2 и 4. Первый — формализация модели групповых взаиморасчетов с раздельным учетом по валютам, в которой явно зафиксированы инвариант сохранения сальдо и верхняя граница числа взаиморасчетов, формируемая жадным алгоритмом упрощения и не превышающая $N - 1$ переводов на каждую валюту. Второй — верификация денежных инвариантов модели методом тестирования по свойствам классы

входных данных автоматически генерируются по заданным свойствам, что снижает зависимость от полноты ручного набора примеров.

Практическая значимость работы состоит в том, что разработано веб-приложение для совместного учета расходов, применимое для сценариев совместного использования денежных средств. Реализованы добавление трат с равномерным и произвольным распределением долей, автоматический расчет нетто-сальдо участников по валютам, упрощение взаиморасчетов до списка не более $N - 1$ переводов на каждую валюту, фиксация погашений и журнал операций группы.

Реализация выполнена при использовании FastAPI, PostgreSQL, RabbitMQ и React. Контейнеризация через Docker Compose обеспечивает воспроизводимость на одном VPS и создает предпосылки для горизонтального масштабирования stateless-сервисов API-шлюза и клиентского слоя; синглтонные компоненты обработки событий вынесены в направления развития (§3.4).

Результат работы — разработанное веб-приложение для совместного учета расходов. Денежные инварианты предметной области формализованы в виде модели и преобразованы в восемь функциональных и шесть нефункциональных требований. Серверная часть реализована на FastAPI с разделением на сервисы аутентификации, групп, расходов и уведомлений; данные хранятся в схемах одного экземпляра PostgreSQL. Реализован жадный алгоритм упрощения взаиморасчетов с верхней границей $N - 1$ переводов на каждую валюту.

Раздел 1 «Теоретические основы и проектирование» содержит анализ предметной области, сравнительный анализ существующих решений, обоснование архитектурного подхода и технологического стека, проектирование архитектуры системы и схемы базы данных, формализацию финансовой модели групповых взаиморасчетов.

Раздел 2 «Реализация и тестирование» описывает реализацию серверной части на FastAPI, клиентской части на React, инфраструктуры виртуализации, функциональное и инвариантное тестирование с оценкой соответствия требованиям.

Раздел 3 «Анализ результатов и направления развития» содержит анализ принятых архитектурных решений, сопоставление с альтернативными подходами, оценку эффективности реализованного решения и систематизацию направлений развития системы.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И ПРОЕКТИРОВАНИЕ СИСТЕМЫ

1.1 Анализ предметной области

Под совместными расходами понимаются затраты, понесенные одним или несколькими участниками заранее определенной группы в интересах всей группы и подлежащие распределению между ее участниками в установленных долях.

Совместный учет расходов отличается от смежных классов задач — персонального бюджета [5], электронной коммерции и дистанционного банковского обслуживания. Рассматриваемый класс систем оперирует многосторонними обязательствами и согласованным представлением данных траты у всех ее участников.

Профильные решения для многосторонних групповых расчетов с открытым исходным кодом и полноценной локализацией на русский язык на российском рынке представлены ограниченно. Сравнительный анализ существующих систем приводится в §1.2.

Для строгого описания введем базовые понятия предметной области и связывающие их соотношения.

Группа. Конечное непустое множество участников $U = \{u_1, u_2, \dots, u_n\}$, в рамках которого ведется совместный учет расходов.

- *Владелец группы* — пользователь, создавший группу. Он наделен полным набором прав по управлению ее составом и параметрами: приглашение и удаление участников, изменение свойств группы, ее закрытие.
- *Участник группы* — пользователь, принявший приглашение. Он добавляет траты, фиксирует платежи в счет погашения задолженности, просматривает балансы и историю операций.

- *Приглашенный пользователь* — лицо, получившее ссылку-приглашение, но еще не принявшее ее и до принятия не имеющее доступа к данным группы.
- *Групповые путешествия*. Участники поочередно оплачивают общие расходы на проживание, транспорт, питание и развлечения. Сценарий отличают высокая интенсивность операций в течение ограниченного периода и необходимость окончательного урегулирования по его завершении. Для него характерны группы малого и среднего размера.
- *Совместное проживание*. Сожители и арендаторы регулярно несут общие расходы на аренду, коммунальные услуги, бытовые покупки и продукты. Сценарий отличают длительный жизненный цикл группы, регулярный характер части расходов, присутствие постоянных и эпизодических участников. Он формирует основную долговременную нагрузку на систему.
- *Корпоративные и социальные мероприятия*. Коллективные обеды, поздравления и совместные подарки: один участник единовременно оплачивает счет за всех с последующим распределением суммы. Характерны эпизодические группы, переменный состав и преобладание одной траты на группу.

Трата. Запись об оплате суммы S участником-плательщиком с распределением обязательств между подмножеством участников $U' \subseteq U$. Доля участника u_i обозначается a_i и выражается в тех же денежных единицах, что и сумма S . Доли удовлетворяют условию сохранения полной суммы траты, которое записывается по формуле

$$\sum_{u_i \in U'} a_i = S \quad (1.1)$$

где U' — подмножество участников, между которыми распределяется трата; u_i — участник из U' ; a_i — доля участника u_i ; S — полная сумма

траты. Формула (1.1) выражает финансовую непротиворечивость: сумма обязательств по одной трате равна фактически уплаченной сумме. Неотрицательность долей исключает превращение участника в получателя средств внутри одной траты, а нулевая доля допускает присутствие участника без обязательства.

Функциональная модель системы включает пять базовых операций.

1. *Добавление траты.* Фиксируется плательщик, общая сумма, дата, описание, категория и состав участников, между которыми распределяется расход.
2. *Распределение суммы.* Сумма делится между участниками поровну либо в произвольных явно заданных долях.
3. *Расчет балансов.* Для каждого участника вычисляется суммарный баланс — разность между суммой, уплаченной им за других, и суммой, которую он должен другим. Положительный баланс соответствует нетто-кредитору группы, отрицательный — нетто-должнику.
4. *Формирование взаиморасчетов.* На основании балансов формируется набор направленных переводов, обнуляющих обязательства всех участников. Точная минимизация числа переводов NP-трудна, поэтому применяется приближенная жадная стратегия (§1.6) [6, 7].
5. *Погашение задолженности.* Участник фиксирует произведенный платеж в пользу другого участника, после чего система пересчитывает балансы.

Совместная трата проходит жизненный цикл из последовательных этапов: создание записи с указанием плательщика, суммы и долей; редактирование с перерасчетом балансов затронутых участников; учет в балансах до урегулирования.

Баланс участника u_i — алгебраическая сумма его финансовых обязательств в группе. Пусть E_i^+ — множество трат, где участник u_i был плательщиком, а E_i^- — множество трат, в распределении которых участник

u_i входит в состав должников. Тогда баланс участника вычисляется по формуле

$$b(u_i) = \sum_{e \in E_i^+} S_e - \sum_{e \in E_i^-} a_{ie} \quad (1.2)$$

где b_i — баланс участника u_i ; E_i^{+} — множество трат, оплаченных участником u_i ; E_i^{-} — множество трат, где участнику u_i назначена доля; S_e — полная сумма траты e ; a_{ie} — доля участника u_i в трате e .

Долговой граф группы. непогашенные обязательства представляются ориентированным графом $G = (V, E)$: вершины V соответствуют участникам U , а каждое ребро (u_i, u_j) соответствует чистой задолженности участника u_i перед участником u_j с весом, равным размеру обязательства. Понятие применяется при описании минимизации взаиморасчетов и в §1.2, где соответствующий алгоритмический класс именуется упрощением долгового графа.

Взаиморасчет. Конечный набор направленных переводов t_{ij} от участника u_i к участнику u_j , обнуляющих балансы всех участников; t_{ij} — сумма перевода от u_i к u_j . Взаиморасчет интерпретируется как замена исходного долгового графа эквивалентным по балансам графом с по возможности меньшим числом ребер. Для каждого участника u_i условие баланс-нейтральности задается формулой

$$\sum_j t_{ji} - \sum_j t_{ij} = b(u_i) \quad (1.3)$$

где t_{ji} — сумма входящего перевода от участника u_j к участнику u_i ; t_{ij} — сумма исходящего перевода от участника u_i к участнику u_j ; b_i — баланс участника u_i , вычисленный по формуле (1.2). Формула (1.3) формализует закрытие всех обязательств: положительный баланс кредитора покрывается превышением входящих переводов над исходящими, отрицательный баланс должника — чистым оттоком. Условие выполняется одновременно для всех участников группы.

Из формул (1.2) и (1.3) следует характеристический инвариант: сумма балансов всех участников тождественно равна нулю на любой стадии работы системы. Он выражает закон сохранения денежных средств в учетной модели — каждое обязательство одного участника есть встречное обязательство другого.

Корректная реализация модели требует инвариантов, не выражаемых арифметикой над вещественными числами: хранение сумм в минимальных денежных единицах (копейках) в целочисленной арифметике, устраняющее погрешность формата с плавающей точкой; детерминированное правило распределения остатка при равном делении; идемпотентность пересчета баланса при повторном редактировании или удалении траты.

Задача минимизации мощности множества переводов при выполнении условия формулы (1.3) для каждого участника в общем случае NP-трудна [7] и в практических реализациях решается приближенными жадными стратегиями [3, 6] (§1.6).

Функциональные и нефункциональные требования

Из анализа предметной области выводятся функциональные требования к системе.

- *F1.* Регистрация и авторизация пользователей с защищенным хранением учетных данных.
- *F2.* Создание групп и управление их составом по пригласительной модели вступления.
- *F3.* Добавление, редактирование и удаление трат с указанием плательщика, суммы, описания и состава участников.
- *F4.* Два режима распределения суммы — равное деление и распределение по явным долям.
- *F5.* Автоматический расчет балансов всех участников по формуле (1.2).
- *F6.* Формирование взаиморасчетов, удовлетворяющих формуле (1.3).

Реализованы оба варианта представления: попарная нормализация

встречных задолженностей и жадная минимизация по нетто-сальдо до числа переводов не более $N - 1$ на каждую валюту (§1.6).

- *F7*. Регистрация платежей в счет погашения задолженности с обновлением балансов.
- *F8*. Хранение и отображение истории операций группы.

Из анализа сценариев и масштаба практики формируются нефункциональные требования.

- *NF1*. Финансовая корректность — соблюдение инвариантов формул (1.1), (1.2), (1.3) и обнуления совокупного баланса на любой стадии работы системы.
- *NF2*. Веб-интерфейс, доступный с настольных и мобильных устройств без установки нативного приложения.
- *NF3*. Локализация на русский язык и адаптация под российскую аудиторию.
- *NF4*. Воспроизводимость запуска и масштабируемость архитектуры с учетом роста числа пользователей.
- *NF5*. Защищенность персональных и финансовых данных.
- *NF6*. Полнота журналирования операций для аудита и восстановления состояния группы.

Эти требования служат основой для сравнительного анализа решений в §1.2 и обоснования архитектурного и технологического выбора в §1.3.

Практика совместного учета расходов в современной России охватывает значительные группы пользователей и поддерживается тремя устойчивыми сценариями — групповыми путешествиями, совместным проживанием и коллективными мероприятиями [1, 2].

В предметной области выделены три роли пользователей и пять базовых операций, охватывающих весь жизненный цикл совместной траты. Задача минимизации числа переводов относится к классу NP-трудных и решается приближенными жадными алгоритмами [6, 7].

Требования F1–F8 и NF1–NF6 задают функциональный профиль, наличие которого проверяется в существующих решениях: в §1.2 каждая система сопоставляется с этим набором, а выявленные пробелы используются в §1.3 при обосновании архитектурного и технологического выбора.

1.2 Обзор и сравнительный анализ существующих решений

Требования, выведенные в §1.1, проверяются далее в существующих зарубежных и российских решениях для выявления их возможностей, ограничений и применимости к задачам работы.

Для сравнения отобраны системы разных сегментов: специализированные трекары совместных расходов, персональные финансовые менеджеры с элементами коллаборации и встроенные инструменты разделения платежей в приложениях российских банков. Критерии обусловлены требованиями §1.1 и практической значимостью открытого локализованного варианта.

Splitwise — наиболее распространенный зарубежный сервис учета совместных расходов, основанный в 2011 году [3]. Ключевая особенность — встроенный алгоритм упрощения долгового графа Simplify Debts, сводящий задолженности к сокращенному набору переводов между нетто-должниками и нетто-кредиторами; его устройство разбирается в §1.3.1 [8].

Tricount — бельгийское мобильное приложение, в 2022 году приобретенное нидерландским банком bunq [4], ориентированное на временные группы. Групповые расчеты реализованы в полном объеме (распределение поровну или в долях, балансы, итоговые взаиморасчеты); алгоритм минимизации присутствует, но публично не документирован.

Settle Up — трекер чешской Step Up Labs [9] для iOS, Android и браузера.

Splid — мобильное приложение команды Team Turtle для iOS, Android и веба [10] с упрощенной моделью работы: регистрация не требуется, группа

создается локально и при необходимости синхронизируется по ссылке, поддерживается офлайн-режим. Групповые расчеты реализованы полностью (распределение поровну, в долях и с несколькими плательщиками, более 150 валют с автоконвертацией, экспорт в PDF и Excel); алгоритм минимизации не документирован.

Дзен-мани — российский сервис учета личных финансов для индивидуальных пользователей [11] с категоризацией доходов и расходов, автоматическим импортом банковских операций и отчетами по бюджету. Русскоязычный интерфейс и интеграция с российскими банками выгодно отличают его от зарубежных решений, однако многосторонние групповые расчеты не поддерживаются: создание групп, распределение трат и расчет взаимных балансов отсутствуют.

CoinKeeper — российский инструмент управления личным бюджетом с визуализацией расходов в форме «монет» [12], поддерживающий ручной ввод и импорт выписок, категоризацию, планирование и отчеты. Тариф Platinum допускает совместное ведение единого бюджета с разных устройств, но это синхронизация общего набора счетов между членами семьи, а не учет многосторонних обязательств между независимыми участниками: создать группу с разделением долей, внести общую трату и получить список взаиморасчетов нельзя.

Т-Банк реализовал функцию «Разделить»: из записи об операции пользователь указывает участников из контактов, и каждому отправляется запрос на перевод на карту [13]. Сбербанк в СберБанк Онлайн предоставляет функцию «Сборы»: организатор создает целевой сбор или распределяет сумму поровну между участниками, которым отправляется push-приглашение присоединиться к сбору [14].

В таблице 1 рассмотренные системы сопоставлены по ключевым критериям.

Таблица 1 — Сравнительный анализ существующих решений для учета совместных расходов

Критерий	Splitwise	Tricount	Settle Up	Splid	Дзен-мани	CoinKeeper	Т-Банк / Сбербанк
Групповые расчеты	Да	Да	Да	Да	Нет	Нет	Частично (одна транзакция)
Алгоритм упрощения взаиморасчетов (сокращение числа переводов)	Да [8]	Да*	Да**	Да*	Нет	Нет	Нет
Открытый исходный код	Нет	Нет	Нет	Нет	Нет	Нет	Нет
Русскоязычная локализация	Нет	Нет	Да	Нет	Да	Да	Да
Модель монетизации	Freemium	Бесплатно	Freemium	Разовый платеж	Freemium	Freemium	Бесплатно (в рамках банка)
Самостоятельный запуск на собственной инфраструктуре	Нет	Нет	Нет	Нет	Нет	Нет	Нет

* алгоритм заявлен разработчиками сервиса, однако его реализация публично не документирована.

** алгоритм заявлен, реализация не документирована публично [9].

Как следует из таблицы 1, ни одно решение не удовлетворяет всем требованиям одновременно. Settle Up закрывает языковой разрыв при полноценной функциональности, сохраняя те же ограничения.

Проведенный анализ позволяет сформулировать следующие выводы.

Во-первых, ни одна из рассмотренных систем не совмещает групповые расчеты с минимизацией взаиморасчетов, открытый код, русскоязычную локализацию и самостоятельный запуск приложения. Splitwise и Tricount технически наиболее близки к требуемому профилю, но недоступны для адаптации и самостоятельный запуск из-за закрытости кода; отечественные аналоги решают иную задачу.

Во-вторых, ни один из рассмотренных сервисов не предоставляет возможность самостоятельного запуска: данные участников располагаются в облаке поставщика, что делает продукты неприменимыми при повышенных требованиях к приватности и ограниченном доступе к внешним облачным сервисам. Коммерческие модели этой зависимости не снимают: freemium у Splitwise, Дзен-мани и CoinKeeper и разовый платеж у Splid лишь дополняют ее.

В-третьих, Settle Up показывает, что русскоязычный трекер групповых расходов существует, но сохраняет закрытый код и отсутствие самостоятельного запуска. Неудовлетворенная потребность — открытое решение с самостоятельным размещением и поддержкой русского языка; эта комбинация свойств определяет задачу работы.

Совокупность ограничений обосновывает разработку собственной системы, удовлетворяющей следующим требованиям, согласованным с §1.1.

- Полноценные групповые расчеты с гибкими схемами распределения трат (F3, F4).
- Встроенный алгоритм взаиморасчетов: нормализация попарных встречных задолженностей и жадная минимизация по нетто-сальдо до числа переводов не более $N - 1$ (F6, реализация — §1.6).

- Открытый исходный код для изучения, модификации и независимого аудита.
- Русскоязычный интерфейс, адаптированный для российской аудитории (NF3).
- Запуск на произвольной инфраструктуре без зависимости от внешних облачных провайдеров (NF4).

Помимо перечисленного, система должна обеспечивать полный набор требований §1.1 — аутентификацию и управление составом групп (F1, F2), расчет балансов и регистрацию погашений (F5, F7), хранение истории (F8), финансовую корректность (NF1), доступность веб-интерфейса (NF2), защищенность данных (NF5) и полноту журналирования (NF6).

1.3 Архитектурный анализ аналогов и обоснование технологического стека

Функциональное сравнение в §1.2 обосновало необходимость собственной разработки, но оставило открытым вопрос, какие инженерные практики стоят за рассмотренными продуктами и какие из них применимы к проектируемой системе. Те же системы рассматриваются далее со стороны стека и инженерных решений.

Источники различаются по раскрытости: одни компании публикуют технологический блог и open-source библиотеки, другие распространяют только продукт.

1.3.1 Архитектуры

Splitwise серверная часть построена на Ruby on Rails в гибридной инфраструктуре AWS и Heroku, что зафиксировано в описании вакансии backend-инженера [15] и подтверждено инженерным блогом с заметками об ускорении расчета балансов [16].

Алгоритмическое ядро — функция Simplify Debts — опирается на классические методы теории графов: расчет сводится к вычислению

максимального потока на остаточном графе по нетто-сальдо участников с последовательным удалением насыщенных путей [8]. Близкий выбор жадной стратегии по net balance с категоризацией участников представлен и в открытых публикациях [17].

Инженерный блог Splitwise фиксирует прием масштабирования расчета: тяжелую агрегацию балансов отделяют от обработки запросов и кешируют ее результат, поскольку при росте истории расходов прямой расчет по операционным таблицам становится узким местом [16]. В проектируемой системе кеширование агрегата отнесено к версии v2 (снимок balance_snapshot, §1.4.4); в MVP балансы вычисляются по запросу из ledger-таблиц. Клиентская часть Splitwise публично не документирована, поэтому в архитектурном сравнении рассматривается только серверная сторона.

Описан переход с AppEngine на Realtime Database, реактивная синхронизация через подписки и решение задачи офлайн-режима [18]. Канал доставки уведомлений — Firebase Cloud Messaging, управляемый web-push-сервис [19].

Web-push как канал уведомления — индустриально проверенное решение; в проектируемой системе оно реализовано через собственный notify-service, хранящий подписки в push_subscriptions и фиксирующий доставку в notification_deliveries (§1.5.5), что воспроизводит результат Settle Up без зависимости от Firebase.

Zenmoney. утилита zenmoney-backup экспортирует данные в JSON через OAuth-токен и демонстрирует устойчивые контракты запросов и формат сущностей [20]. Свиридовой отмечено, что импорт через банковский API надежнее парсинга SMS [21].

CoinKeeper. Проект mk-manishkumar/coinkeeper — pet-project, не связанный с российским сервисом CoinKeeper [12], — опубликован с явным указанием стека: фронтенд на React и TypeScript с Tailwind CSS и Redux

Toolkit, бэкенд на Node.js и Express с MongoDB, аутентификация через JWT и bcryptjs [22].

По Tricount, Splid и банковским инструментам (Т-Банк, Сбербанк) публичных инженерных материалов об архитектуре нет, поэтому в §1.3 они не разбираются и остаются в функциональном сравнении §1.2.

1.3.2 Технологический стек разрабатываемой системы

Рассмотренные архитектуры дают три точки сопоставления: серверный язык и фреймворк, способ хранения данных и канал доставки событий, клиентский стек и режим рендеринга. По каждой принят набор технологий, опирающийся на индустриальную практику и устраняющий выявленную в §1.2 зависимость от чужой инфраструктуры.

Серверная часть: FastAPI, Python, PostgreSQL, RabbitMQ. Серверным фреймворком принят FastAPI поверх ASGI-каркаса Starlette и библиотеки валидации Pydantic v2.

В сравнении с Ruby on Rails (Splitwise) и Node.js/Express (CoinKeeper-подобное приложение [22]) Python с FastAPI выбран по трем соображениям: встроенный decimal для арифметики с фиксированной точкой над минимальными денежными единицами (§1.6); декларативная валидация на Pydantic, снимающая рассинхронизацию схем и документации API; нативная поддержка async/await.

Реляционной СУБД принята PostgreSQL 16, что соответствует литературе по проектированию баз с нетривиальной структурой агрегатов [23]. Логическое разделение сервисов обеспечивают схемы auth, groups, expenses, notify в одном экземпляре базы, что сохраняет переносимость кода к отдельной БД на сервис. Миграции выполняются через Alembic поверх SQLAlchemy.

Брокером сообщений выбран RabbitMQ. По сравнению с журнально-ориентированными брокерами (log-based) RabbitMQ потребляет меньше

ресурсов и разворачивается в одном контейнере, что соответствует ограничениям одного VPS.

Клиентская часть: React, Vite, TanStack Query, Zustand. Базовой библиотекой принят React 19. Выбор опирается на open-source expense-tracker уровня CoinKeeper с React, TypeScript и Tailwind CSS [22] и на распространение Splitwise как SPA поверх Rails-API [3, 16]. Сравнение с Vue.js не проводится: аналоги используют React либо Angular в составе Firebase.

Управление состоянием разделено по природе данных: серверное состояние изолировано в TanStack Query, клиентское состояние контекста аутентификации — в Zustand.

Сборщиком выбран Vite с tree-shaking над ESM-модулями и автоматическим code splitting: маршруты выносятся в отдельные чанки, и первичный бандл не включает код экранов, недоступных без аутентификации.

Пограничным сервером применяется Nginx с асинхронной моделью обработки соединений. Все компоненты упаковываются в Docker-контейнеры и описываются в Docker Compose как единая топология с общими сетями и именованными томами для PostgreSQL и RabbitMQ, запуск на новом сервере сводится к одной команде.

Данный подход также выбран в связи с тем, что это исследовательский проект, и его запуск будет ограничен одним виртуальным сервером.

Принятый стек собран из решений, применимость которых подтверждается архитектурами Splitwise (кеширование агрегатов поверх реляционной СУБД, SPA-клиент) и Settle Up (web-push как канал уведомлений), а также open-source expense-tracker-приложениями с React-фронтом на TypeScript. Отличие сосредоточено в инфраструктуре: вместо закрытой облачной инфраструктуры Splitwise и Firebase принят набор самостоятельно размещаемых компонентов — PostgreSQL, RabbitMQ, Nginx

— в Docker Compose, что устраняет выделенную в §1.2 зависимость от внешнего провайдера.

Архитектурный анализ показал, что специализированные трекары опираются на зрелые серверные стеки (Ruby on Rails у Splitwise, Firebase у Settle Up) и предсказуемые клиентские решения (SPA у Splitwise, Angular у Settle Up), а отечественные финансовые приложения формируют вокруг себя экосистему открытых API (Дзен-мани) при закрытой серверной реализации.

Принятый стек сочетает FastAPI на Python с PostgreSQL и RabbitMQ для сервера, React 19 с Vite, TanStack Query и Zustand для клиента, Nginx и Docker Compose для инфраструктуры. Детальное проектирование архитектуры выполняется в §1.4.

1.4 Проектирование архитектуры системы

На основе технологических решений §1.3 ниже проектируется компонентная архитектура: состав сервисов и границы их ответственности, схема аутентификации на базе JWT RS256, событийная архитектура с паттерном Outbox и стратегия консистентности групповых балансов [24].

1.4.1 Компонентная архитектура системы

Каждый сервис разворачивается отдельным контейнером средствами Docker Compose.

Nginx очищает входящие заголовки семейства X-User- — прием защиты от подделки идентификационных данных в gateway-pattern микросервисной архитектуры [25].

Api-gateway — доверенная точка формирования контекста идентификации: проверяет токен в заголовке Authorization: Bearer, валидирует JWT по публичному ключу auth-service и при успехе формирует заголовки X-User-* с идентификатором и ролью пользователя, после чего проксирует запрос к backend-сервису.

Frontend реализует интерфейс как одностраничное приложение с клиентским рендерингом; статический бандл отдается через Nginx.

Auth-service отвечает за регистрацию и аутентификацию, выдачу пар токенов, ротацию семейства refresh-токенов и отзыв токенов при выходе.

Groups-service отвечает за создание групп, управление участниками, приглашения и назначение ролей. Сервис — единственный источник истины о членстве; его схема groups содержит таблицы групп, участников и приглашений.

Expenses-service реализует основную бизнес-логику: хранение расходов, правила распределения, расчет балансов, взаиморасчеты (settlement), журнал аудита критичных действий и генерацию доменных событий. Ключевые модули: split_engine — разбиение суммы расхода; balance_engine — on-demand расчет балансов из ledger-таблиц; outbox — таблица исходящих событий для асинхронной доставки уведомлений.

Expenses-worker разворачивается отдельным контейнером из образа expenses-service и публикует сообщения: читает записи expenses.outbox со статусом pending, публикует их в RabbitMQ exchange app.events типа topic и повторяет доставку при временных сбоях.

Notify-service подписан на очередь RabbitMQ, идиоматично обрабатывает доменные события и формирует уведомления для участников. Доставка идет по двум активным каналам — in-app (сохраняемая запись) и web-push, — факт доставки фиксируется в таблице notify.notification_deliveries, резервирующей и третий канал, email.

Взаимодействие компонентов представлено на диаграмме (Рисунок 1): сплошные стрелки — синхронные HTTP-соединения, асинхронные зависимости идут через брокер, пунктирная стрелка выделяет прямое обращение expenses-service к таблице членства. Подписью v2 обозначены элементы, которые вынесены в направления развития (§3.4).

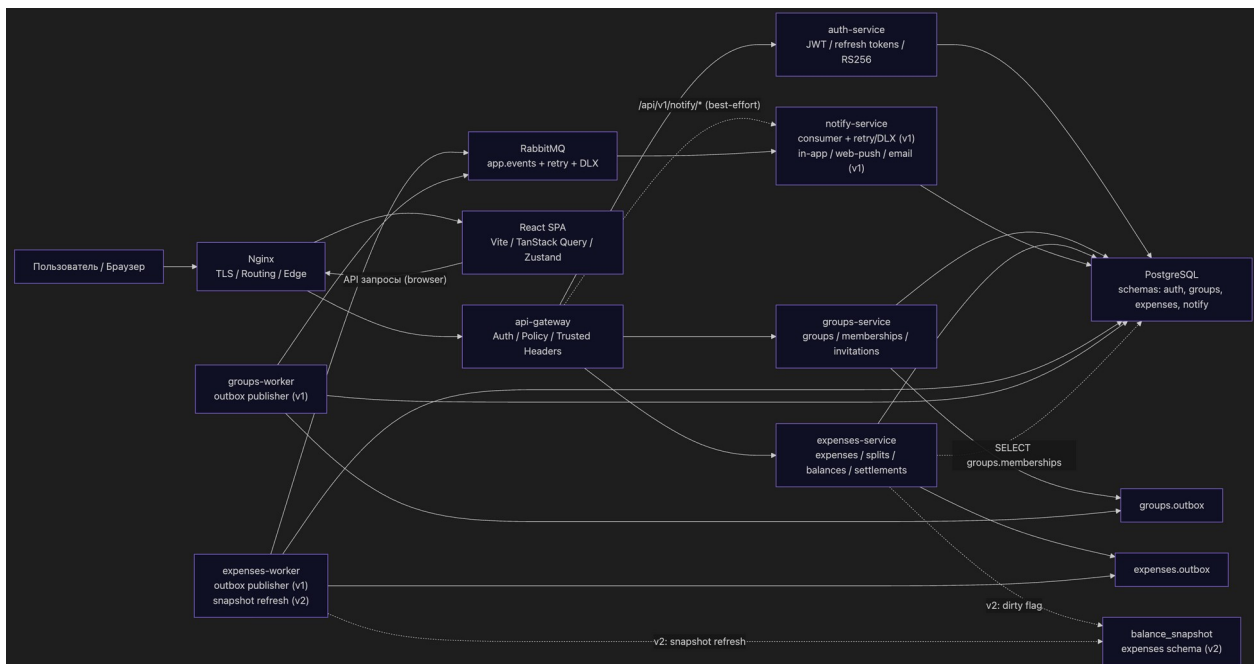


Рисунок 1 — Компонентная архитектура системы

Разбиение на сервисы допускает независимое масштабирование и самостоятельный запуск. При недоступности notify-service операции с расходами и балансами продолжают работать: события накапливаются в outbox и доставляются после восстановления брокера.

1.4.2 Схема аутентификации и безопасности

Безопасность строится на нескольких рубежах защиты от несанкционированного доступа.

Состав claims access token соответствует стандарту RFC 7519 [26]: sub (идентификатор пользователя), iss (issuer), aud (audience), iat/nbf/exp (временные метки выдачи, начала действия и истечения срока), jti (уникальный идентификатор токена, используемый для детектирования повторного использования).

Access token имеет короткий срок действия, что ограничивает временное окно компрометации при его перехвате. Refresh token выдается на более длительный срок и хранится в защищенном HTTP-only cookie, недоступном для JavaScript-кода клиентского приложения.

Все запросы, поступающие к backend-сервисам, проходят через api-gateway, который выступает единственной доверенной точкой формирования контекста идентификации.

Ролевая модель доступа. Внутри каждой группы пользователям назначается одна из трех ролей: owner (создатель группы, полный контроль), admin (управление участниками и расходами), member (просмотр и добавление расходов). Проверка принадлежности пользователя к группе и его роли выполняется на уровне expenses-service и groups-service.

Дополнительные меры защиты. В production-конфигурации (nginx.prod.conf) на endpoints аутентификации применяется rate limiting средствами Nginx для /auth/login установлено ограничение 5 запросов в минуту, для /auth/refresh — 30 запросов в минуту, что существенно ограничивает возможность атак методом перебора учетных данных.

1.4.3 Событийная архитектура

Уведомление участников о финансовых событиях реализовано асинхронно на базе брокера RabbitMQ. Требование к компоненту — надежная доставка событий при транзакционной консистентности с основной операцией: уведомление отправляется тогда и только тогда, когда операция зафиксирована в базе данных.

Паттерн Outbox. В соответствии с принятым архитектурным решением (§1.3) для обеспечения транзакционной консистентности применяется паттерн Outbox [27]: при создании расхода или проведении взаиморасчета в рамках одной атомарной транзакции PostgreSQL выполняются два действия — сохранение основных данных операции и запись соответствующего доменного события в таблицу outbox со статусом pending.

Такой подход обеспечивает семантику доставки «не менее одного раза» [28].

Routing keys событий соответствуют типу выполненной операции: `expense.created`, `expense.updated`, `expense.deleted` — при создании, изменении и удалении расхода, `settlement.created` — при проведении взаиморасчета. Использование топологии `topic` позволяет потребителям подписываться как на конкретные типы событий, так и на их группы по маске.

Notify-service и идемпотентная обработка. `notify-service` подписан на очередь, привязанную к `exchange app.events`. Такая схема обеспечивает эффект однократной обработки (`effectively-once`) на прикладном уровне: транспорт сохраняет семантику `at-least-once`, а повторные доставки отсекаются проверкой идентификатора события в `processed_events` [29].

Перенаправление в DLQ снимает блокировку основной очереди и удерживает поврежденные сообщения для отдельного разбора [28].

Схема разделяет `expenses-service` и `notify-service`: учет расходов не зависит от доступности сервиса уведомлений, а при недоступности `notify-service` события удерживаются в очереди RabbitMQ и обрабатываются после восстановления соединения.

1.4.4 Стратегия консистентности балансов

Расчет балансов агрегирует данные расходов, долей и взаиморасчетов. Реализован двухэтапный подход, соответствующий стадиям развития продукта.

Вычисление балансов по запросу. Баланс участника вычисляется в момент обращения к эндпоинту из `ledger`-таблиц `expenses`, `expense_splits` и `settlements`:

- суммируется доля участника в расходах — сколько он должен заплатить другим участникам группы;
- вычитается совокупная сумма, фактически им оплаченная за группу по записям с этим `paid_by_user_id`;

— результат корректируется на взаиморасчеты, где пользователь — отправитель или получатель.

Положительный итог означает, что участники суммарно должны пользователю, отрицательный — что должен он.

Упрощение расчетов: greedy-минимизация числа переводов. Дополнительно к попарному представлению реализована greedy-минимизация числа переводов с границей не более $N - 1$, доступная через GET /balances/simplified. Обоснование границы и алгоритм — в §1.6.2.

Подраздел задает архитектурный каркас системы: компонентный состав независимых сервисов, схему аутентификации на JWT RS256 с централизованной проверкой на api-gateway, событийную архитектуру с паттерном Transactional Outbox поверх RabbitMQ и стратегию консистентности балансов через ledger-таблицы с откладыванием balance_snapshot до v2. Принятые решения переводят инварианты §1.1 в проектные ограничения для схемы базы данных и API (§1.5) и реализации серверной части (§2.1).

1.5 Проектирование базы данных и API

Параграф проектирует два уровня контракта системы. СУБД — PostgreSQL: реляционная система с открытым кодом, гарантиями ACID, расширенными типами и механизмом логических схем [30].

1.5.1 Принципы организации хранения данных

Хранение строится на единой физической базе PostgreSQL при логической изоляции сервисов через механизм схем. Каждый backend-сервис владеет собственной схемой и не выполняет записи в чужие. Чтение чужих схем допускается только в задокументированных случаях, где синхронный HTTP-вызов признан неоправданным.

Схема баз данных находится в приложении А.

Из правила изоляции сделаны задокументированные исключения для перекрестного чтения, снимающие синхронные HTTP-вызовы на критических путях: потребителям выдаются SELECT-гранты на чужие схемы. Альтернатива — цепочка синхронных HTTP-вызовов, увеличивающая задержку и связанность доменов, поэтому предпочтено прямое чтение.

База включает четыре логические схемы: auth (пользователи и refresh-токены), groups (группы, членство, приглашения, outbox), expenses (расходы, доли, взаиморасчеты, идемпотентность, outbox, журнал), notify (in-app лента, подписки web-push, журнал доставки, реестр обработанных событий).

1.5.2 Схемы auth и groups

Выбор Argon2id опирается на OWASP Password Storage Cheat Sheet [31]: memory-hard-функция требует значительного объема памяти, что делает массовый перебор экономически нецелесообразным.

Таблица refresh_sessions управляет сессиями. Частичный уникальный индекс uq_auth_refresh_one_active_per_family (§1.5.5) ограничивает число активных сессий в семействе одной записью.

Схема groups описывает социальную структуру приложения. Таблица groups хранит атрибуты группы, включая default_currency_code (код ISO 4217 по умолчанию для новых расходов) и status (active / archived; архив переводит группу в read-only с сохранением истории).

1.5.3 Схема expenses

Схема expenses содержит шесть таблиц, охватывающих жизненный цикл финансовых операций: expenses, expense_splits, settlements, idempotency_keys, outbox, activity_log. Таблица balance_snapshot в v1 отсутствует — балансы вычисляются on-demand из ledger-таблиц; ее введение отнесено к v2 (§1.4.4). Денежная модель определена в §1.6.1.

Таблица `expenses` фиксирует каждый расход группы. Таблица `expense_splits` хранит доли участников, связывая расход (`expense_id`, каскадное удаление) с пользователем и долей `amount_minor`; поле `split_type` (`equal` / `custom`) фиксирует режим формирования доли.

Идемпотентность. Составной первичный ключ (`idempotency_key`, `operation_scope`, `actor_user_id`) исключает коллизии между пользователями и типами операций: `idempotency_key` приходит от клиента в HTTP-заголовке `Idempotency-Key` согласно IETF-драфту [32], `operation_scope` различает тип операции, `actor_user_id` привязывает ключ к пользователю.

Outbox. Таблица `outbox` реализует паттерн `Transactional Outbox` для надежной публикации доменных событий в брокер [29].

Таблица `activity_log` ведет журнал значимых действий (`group_id`, `actor_user_id`, `entity_type`, `entity_id`, `action_type`, `data` типа `JSONB`). `Append-only`-режим обеспечивается сервисным слоем `expenses-service`: операции `UPDATE` и `DELETE` в коде не предусмотрены. Усиление запрета через `REVOKE` для роли `expenses_user` отнесено к `production`-конфигурации.

1.5.4 Схема notify

Схема `notify` обслуживает сервис уведомлений и содержит четыре таблицы: представление уведомления, реестр транспортных эндпоинтов, журнал доставки и реестр идемпотентности обработки событий. Таблица `notifications` реализует ленту входящих уведомлений (`in-app`), где каждая запись соответствует одному уведомлению адресату.

Таблица `push_subscriptions` хранит подписки на `web-push` (`endpoint` с ограничением `UNIQUE`, криптографические параметры `p256dh` и `auth`, счетчик `failure_count` для деактивации при коде 410 `Gone`). Механизм реализует семантику `exactly-once` на прикладном уровне при гарантии `at-least-once` со стороны брокера [29].

1.5.5 Инварианты данных

Корректность данных поддерживается двухуровнево. Средствами PostgreSQL зафиксированы: положительность сумм (CHECK amount_minor > 0 на expenses и expense_splits); запрет самозачета (CHECK from_user_id <> to_user_id на settlements); уникальность участника в группе (UNIQUE на memberships(group_id, user_id)); идемпотентность обработанного события (PK (event_id, consumer_name) на processed_events); единственность активной сессии в семействе токенов (частичный uq_auth_refresh_one_active_per_family на refresh_sessions(token_family_id) WHERE status='active', снимает гонку при параллельных /auth/login и /auth/refresh, при которой детектирование компрометации (§1.5.2) становится неоднозначным); замкнутость домена статуса сессии (CHECK, шесть значений); уникальность пары «событие — получатель» (UNIQUE (event_id, user_id) на notifications); уникальность транспортного эндпоинта web-push (UNIQUE endpoint на push_subscriptions); ссылочная целостность внутри схем — внешними ключами с явными ON DELETE.

На уровне приложения в транзакции создания проверяются равенство суммы долей сумме расхода, активное членство участников через SELECT к groups.memberships (задокументированное исключение изоляции), действие инициатора погашения в своей группе и наличие задолженности, на которую ссылается settlement.

1.5.6 Стратегия миграций

Схема версионруется инструментом Alembic [33] — стандартным решением в экосистеме SQLAlchemy [33, 34]. Каждая миграция содержит upgrade() и downgrade(), обеспечивая воспроизводимость развертывания и откат; декларативные ORM-модели служат источником истины для автогенерации (alembic revision --autogenerate) [34].

ER-диаграммы базы данных приведены в А: первая охватывает схемы auth и groups, вторая — expenses и notify; разбиение вызвано числом таблиц и связей, при котором единая диаграмма стала бы нечитаемой.

1.5.7 Принципы проектирования API

Программный интерфейс строится на стиле REST: ресурсы идентифицируются URI, взаимодействие выполняется методами GET, POST, PATCH, DELETE, ответы содержат стандартные коды состояния. Контракт описывается в формате OpenAPI 3.1 [35]: в FastAPI схемы генерируются из аннотаций типов и Pydantic-моделей, исключая рассинхронизацию кода и документации [36], а спецификация доступна по /docs и /openapi.json.

Идемпотентность мутирующих операций опирается на таблицу expenses.idempotency_keys (§1.5.3): создание расхода и запись погашения принимают заголовок Idempotency-Key, повтор с тем же ключом и телом возвращает кэшированный ответ с кодом 200 OK, с тем же ключом, но другим телом — 409 Conflict.

Система предоставляет четыре группы ресурсов согласно сервисной декомпозиции: auth-service управляет учетными записями и сессиями (девять эндпоинтов под /api/v1/auth), groups-service — жизненным циклом групп, членством и приглашениями, expenses-service — финансовым ядром, notify-service — лентой уведомлений и подписками web-push. Показательные эндпоинты приведены ниже; полный перечень маршрутов — в Приложении Б.

- POST /api/v1/auth/login — аутентификация: access-токен JWT RS256 (15 минут) возвращается в теле, refresh-токен — в HTTP-only Secure Cookie с SameSite=Strict;
- POST /api/v1/groups — создание группы: в одной транзакции создается запись группы и членство инициатора с ролью owner;

- POST /api/v1/expenses — создание расхода: после проверки инвариантов (§1.5.5) в одной транзакции пишутся expenses, expense_splits, outbox; поддерживается Idempotency-Key;
- GET /api/v1/expenses/{group_id}/balances/simplified — упрощенный список переводов длиной не более $N - 1$ (N — число участников с ненулевым балансом), вычисляется ledger-алгоритмом из §1.6;
- GET /api/v1/expenses/{group_id}/history — журнал группы с offset-based пагинацией (limit, offset, next_offset); курсорная пагинация по (created_at, id) — v2.

Подраздел определяет двухуровневый контракт системы: на уровне хранения и на уровне программного интерфейса. Четыре логические схемы PostgreSQL (auth, groups, expenses, notify) с отдельными ролями и SELECT-грантами, статусная модель outbox с двухфазным claim'ом, инварианты данных на уровне CHECK-ограничений и стратегия миграций через Alembic формируют слой хранения. REST API в спецификации OpenAPI 3.1 с автогенерацией контрактов из Pydantic и идемпотентностью через заголовок Idempotency-Key формирует интерфейсный слой. Эти решения создают основу для реализации серверной части (§2.1) и сохраняют переносимость к отдельным базам на сервис при росте нагрузки.

1.6 Финансовая модель

Подраздел формализует финансовую модель, работающую на двух согласованных уровнях вычислений. На первом уровне попарный журнал финансовых операций строит представление состояния долгов из расходов и зафиксированных погашений, обеспечивая инварианты сохранения суммы расхода и аддитивности баланса (формулы (1.1)–(1.2)). На втором уровне жадный алгоритм по нетто-сальдо вырабатывает список переводов, удовлетворяющий условию баланс-нейтральности (формула (1.3)), с верхней границей $N - 1$ переводов на каждую валюту при NP-трудности точной

минимизации [6, 7]. Подразделы 1.6.1 и 1.6.2 раскрывают вычислительную организацию каждого уровня и проверяемые инварианты.

1.6.1 Алгоритм расчета балансов

Финансовая модель наследует денежное представление §1.5.4: суммы хранятся и передаются в поле `amount_minor` целым числом в минимальных единицах валюты (копейки для RUB, центы для USD и EUR). На уровне API `amount_minor` сериализуется в JSON целым числом без промежуточных конвертаций; преобразование в десятичную форму (1234.56) выполняется только на клиенте при отображении.

Состав полей расхода приведен в Таблице 2.

Таблица 2 — Поля доменной модели расхода

Поле	Тип	Описание
<code>paid_by_user_id</code>	UUID	Идентификатор пользователя, фактически оплатившего расход
<code>amount_minor</code>	bigint	Полная сумма расхода в минимальных единицах валюты
<code>currency_code</code>	string (ISO 4217)	Трехбуквенный код валюты расхода
<code>splits[i].user_id</code>	UUID	Идентификатор участника, на которого относится доля
<code>splits[i].amount_minor</code>	bigint	Доля участника в минимальных единицах валюты

Центральным инвариантом модели расхода является условие нормировки долей, задаваемое формулой

$$\sum_{i=1}^n \text{splits}[i].\text{amount_minor} = \text{expense.amount_minor} \quad (1.4)$$

где `splits_i.amount_minor` — доля *i*-го участника расхода в минимальных единицах валюты; `expense.amount_minor` — полная сумма расхода в тех же единицах; *n* — число долей в списке `splits`. Данное условие проверяется перед записью расхода в базу данных; нарушение инварианта влечет возврат ошибки 400 Bad Request с описанием конкретного несоответствия.

Алгоритм split engine реализован на стороне клиентского приложения в составе модуля frontend/src/shared/lib/money.ts и поддерживает два режима распределения суммы:

- Равномерное распределение: сумма расхода делится поровну между участниками. Базовая доля составляет $\text{floor}(\text{expense.amount_minor} / n)$, где n — число участников в списке splits. Остаток в минимальных единицах прибавляется к доле плательщика, что обеспечивает точное выполнение инварианта (формула (1.4)) без ошибки округления.
- Произвольное распределение: пользователь задает каждую долю вручную; клиент при отправке проверяет соответствие суммы долей сумме расхода и блокирует отправку при нарушении инварианта.

В обоих режимах REST API принимает готовый список splits и не формирует доли повторно: сервер проверяет инвариант формулы (1.4) до записи в базу данных, оставаясь арбитром финансовой корректности, а интерактивный пересчет долей в форме создания расхода идет без сетевого обмена.

Балансы участников группы вычисляются по принципу бухгалтерского журнала: каждая зафиксированная операция трактуется как двусторонняя бухгалтерская запись. Алгоритм реализован в функции `_calculate_balances_from_records()` и выполняется в четыре последовательных шага.

Шаг 1. Построение журнала финансовых операций из расходов. Для каждого расхода и каждой его доли, если участник-должник не совпадает с плательщиком, в журнале формируется направленная запись вида «должник → кредитор», задаваемая формулой

$$\text{ledger}[u_{\text{debtor}} \rightarrow u_{\text{creditor}}, \text{currency}] += \text{split.amount_minor} \quad (1.5)$$

где $\text{ledger}[(u_{\text{debtor}}, u_{\text{creditor}}, \text{currency})]$ — запись попарного реестра обязательств по направлению от должника к кредитору в заданной валюте; u_{debtor} — участник, которому назначена доля; u_{creditor} — плательщик

расхода; `split.amount_minor` — доля должника в минимальных единицах валюты. Если плательщик сам является участником расхода, его собственная доля в реестр не включается: пользователь не может быть должником самому себе.

Шаг 2. Применение погашений. Для каждого зафиксированного погашения соответствующая финансовая запись уменьшается на сумму платежа согласно формуле

$$\text{ledger}[u_{\text{from}} \rightarrow u_{\text{to}}, \text{currency}] -= \text{settlement.amount_minor} \quad (1.6)$$

где $\text{ledger}[(u_{\text{from}}, u_{\text{to}}, \text{currency})]$ — запись попарного реестра обязательств по направлению погашения; `settlement.amount_minor` — сумма зафиксированного погашения в минимальных единицах валюты; u_{from} и u_{to} — участники-стороны платежа. Сформированный по шагам 1–2 попарный реестр обязательств служит входом для алгоритма упрощения взаиморасчетов, описанного в §1.6.2.

1.6.2 Алгоритм упрощения взаиморасчетов

После расчета балансов (§1.6.1) набор данных может содержать до $N \cdot (N - 1)$ попарных записей долгов для N участников. Задача сведения произвольного долгового графа к минимальному числу переводов NP-трудна в общем случае [7]; на практике применяется жадный алгоритм с числом транзакций не более $N - 1$ для каждой валюты.

Система предоставляет два согласованных представления одного состояния расчетов.

Попарный реестр обязательств (§1.6.1) эквивалентен вектору нетто-сальдо участников. Нетто-сальдо b_u^{cur} участника u в валюте cur задается формулой

$$b_{\text{cur}}(u) = P(u) - C(u) + S_{\text{out}}(u) - S_{\text{in}}(u), \quad (1.7)$$

где P_u — сумма расходов, оплаченных участником u ; C_u — сумма всех начисленных участнику u долей, включая собственную долю в

оплаченных им расходах; S_{out_u} — сумма отправленных участником u погашений; S_{in_u} — сумма полученных участником u погашений; все величины выражены в $amount_minor$ и ограничены валютой cur .

Для каждой валюты выполняется инвариант сохранения сальдо, задаваемый формулой

$$\sum_u b_{cur}(u) = 0. \quad (1.8)$$

где U_{cur} — множество участников, имеющих операции в валюте cur ; b_u^{cur} — нетто-сальдо участника u в этой валюте. Инвариант выполняется покомпонентно по валютам.

Нетто-сальдо инвариантно относительно нормализации реестра обязательств (§1.6.1): взаимозачет встречных ребер не меняет разности входящих и исходящих сумм. Значение по формуле (1.7) из исходных операций совпадает с сальдо нормализованного реестра и используется далее в жадном алгоритме.

Алгоритм выполняется в три шага. В мультивалютной группе шаги повторяются независимо для каждой валюты: взаимозачет обязательств в разных валютах требовал бы курсов конвертации и выходит за рамки текущей реализации.

Шаг 1. Вычисление сальдо. Для каждого участника в пределах рассматриваемой валюты вычисляется нетто-сальдо по формуле (1.7). Положительное сальдо относит участника к кредиторам, отрицательное — к должникам; участники с нулевым сальдо из дальнейшего расчета исключаются.

Шаг 2. Сортировка. Кредиторы упорядочиваются по убыванию сальдо (наибольший кредитор — первый), должники — по убыванию абсолютного значения долга (наибольший должник — первый).

Пусть N — число участников с ненулевым сальдо в рассматриваемой валюте. Согласно инварианту (1.8), суммарное сальдо оставшихся

участников всегда равно нулю; поэтому когда остается единственный участник, его сальдо обязано быть нулевым, и последний перевод обнуляет сразу двух участников.

Величина — верхняя граница числа переводов, а не гарантия их минимума: точная минимизация NP-трудна [6], и жадная стратегия в отдельных конфигурациях дает число переводов больше оптимального.

Подраздел замыкает теоретическую часть работы формализацией финансовой модели на двух уровнях. Ledger-алгоритм §1.6.1 строит попарное представление долгов из расходов и погашений с инвариантами сохранения суммы расхода и баланс-нейтральности (формулы (1.1)–(1.3)). Жадный алгоритм §1.6.2 преобразует попарный реестр обязательств в список переводов длиной не более $N - 1$ на каждую валюту, удовлетворяющий условию закрытия обязательств при NP-трудности точной минимизации. Формализованная модель и сопровождающие ее инварианты задают требования к серверной арифметике (§2.1) и инвариантному тестированию (§2.4).

Выводы по разделу 1

В §1.1 формализована предметная область: выделены три роли пользователей и пять базовых операций, финансовая модель сведена к инвариантам (1.1)–(1.3) — сохранению суммы траты, аддитивности баланса и баланс-нейтральности набора переводов; задача минимизации числа переводов отнесена к классу NP-трудных. Сформирован профиль требований F1–F8 / NF1–NF6, по которому в §1.2 проведено сопоставление семи продуктов: одновременно открытым, локализованным и самостоятельно размещаемым с поддержкой минимизации не оказался ни один — эта ниша определила задачу работы.

Архитектурный обзор аналогов (§1.3) подтвердил применимость связки «реляционная СУБД + SPA + web-push» и зафиксировал стек: FastAPI с Pydantic v2 для backend, React 19 с Vite, TanStack Query и Zustand для

клиента, PostgreSQL и RabbitMQ как носитель состояния и брокер событий, Nginx и Docker Compose для пограничного слоя. Проектирование архитектуры (§1.4) выделило четыре backend-сервиса (auth, groups, expenses, notify), api-gateway как единственную точку проверки JWT RS256, и связку transactional outbox + RabbitMQ для надежной доставки доменных событий. Refresh-rotation детектирует повторное использование токена; балансы вычисляются по запросу из ledger-таблиц, snapshot отнесен к v2.

В §1.5 описаны четыре логические схемы PostgreSQL под отдельные роли с задокументированными SELECT-грантами для перекрестных чтений, статусная модель outbox с двухфазным claim'ом и watchdog'ом, идемпотентность чувствительных POST'ов по ключу (idempotency_key, operation_scope, actor_user_id) и разграничение ошибок 422 (Pydantic) / 400 (бизнес-валидация) согласно RFC 9110. Финансовое ядро (§1.6) формализовано как ledger-алгоритм с инвариантами (1.4) и (1.8); жадное упрощение долгового графа дает верхнюю границу $N - 1$ переводов на валюту без гарантии точного минимума (NP-трудность).

Предметная модель, требования, архитектура слоев «пограничный — шлюз — доменный», схема БД и контракт API, финансовая модель взаимно согласованы; этот контур переходит в главу 2 как основа для реализации и тестирования.

2 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Реализация серверной части приложения

Подраздел фиксирует, какими модулями, эндпоинтами и таблицами backend воплощает сервисы и инварианты, спроектированные в §1.4–§1.6.

2.1.1 Структура проекта и организация слоев

Серверная часть размещена в backend/services/: шесть исполняемых сервисов — пять доменных (auth, groups, expenses, worker, notify) и инфраструктурный шлюз gateway — плюс общий модуль common. Состав воспроизводит компонентную архитектуру §1.4.1: каждый доменный сервис разворачивается в отдельном контейнере, обслуживает собственную схему PostgreSQL и взаимодействует с другими через HTTP/REST или RabbitMQ.

Внутри каждого сервиса выдержано трехслойное разделение: слой представления реализует FastAPI-роутеры и схемы Pydantic, слой бизнес-логики — доменные операции и инварианты, слой persistence (app/persistence/) содержит ORM-модели SQLAlchemy 2.0 и инфраструктуру сессии.

Общий модуль common устраняет дублирование инфраструктурного кода между сервисами; его состав показан в таблице 3.

Таблица 3 — Состав общего модуля backend/services/common

Файл	Назначение
app_factory.py	Фабрика make_fastapi_app — единообразная инициализация FastAPI и lifespan
health.py	Маршруты liveness (/privat/health) и readiness (/privat/ready)
readiness.py	Класс состояния готовности и запуск schema-guard при старте
schema_guard.py	Сравнение фактической схемы PostgreSQL с метаданными ORM, детекция дрейфа
logging_setup.py	JSON-логирование, middleware привязки request_id
event_types.py	Перечень типов доменных событий
alembic_env.py	Общее окружение для миграций Alembic всех сервисов

2.1.2 Сервис аутентификации

Auth-service реализует схему аутентификации §1.4.2 без отклонений. Подпись JWT — асимметричная RS256 с минимальным набором claims {sub, iss, aud, iat, nbf, exp, jti} без email; приватный RSA-ключ (2048 бит) хранится только в auth-service, а kid (усеченный SHA-256 публичного ключа в PEM) публикуется в JWKS [26], что позволяет api-gateway сопоставить токен с актуальным ключом.

Пароли хранятся как хэш Argon2id: PasswordHasher настроен с time_cost=3, memory_cost=65536, parallelism=4; параметры превышают минимум, рекомендованный OWASP [31]; вычисление и проверка вынесены в пул потоков, чтобы не блокировать цикл событий. Токен формата ref_ с 24 случайными байтами хранится в базе как SHA-256-хэш; ротация семейства токенов реализована в service.py — каждое обновление создает новую запись в auth.refresh_sessions с тем же token_family_id, прежняя переводится в rotated, а предъявление rotated/revoked-токена переводит все семейство в compromised.

2.1.3 Шлюз API (api-gateway)

api-gateway — единственная доверенная точка проверки JWT и формирования контекста идентификации (§1.4.2). Реализация (gateway/app/main.py) построена вокруг обработчика proxy_request, перехватывающего /api/v1/{path} и направляющего запрос к целевому сервису по таблице _target_url.

2.1.4 Сервис групп (groups-service)

groups-service — единственный источник истины о членстве: создание групп, состав участников, приглашения и ролевая модель. Роутер (groups/app/main.py) предоставляет маршруты создания группы

(POST /api/v1/groups), получения списка групп, управления участниками (.../members) и приглашениями.

Приглашение принимает взаимоисключающе `target_user_id` либо `target_email`; при передаче `email` сервис разрешает его в идентификатор уже зарегистрированного пользователя через `get_user_id_by_email` и отвечает 404, если учетной записи нет. Приглашение по `email` с одноразовым токеном для незарегистрированных адресатов вынесено в направление развития (v2), что согласуется со схемой `invitations` (§1.5.3).

Сервис реализует атомарное создание группы с владельцем, трехуровневую ролевую модель с защитой последнего владельца и приглашения по `target_user_id`. Управляемое сервисом членство читается `expenses`-сервисом напрямую при проверке прав на финансовые операции — этот компромисс рассматривается далее.

2.1.5 Сервис расходов (`expenses-service`)

`expenses-service` реализует финансовое ядро §1.6 без отклонений. Роутер (`expenses/app/main.py`) предоставляет эндпоинты создания расхода (POST /api/v1/expenses), погашения (POST .../settlements), попарных балансов (GET .../{group_id}/balances), упрощенных взаиморасчетов (GET .../{group_id}/balances/simplified), истории (GET .../{group_id}/history), а также списка и карточек расходов.

Денежная модель воплощает §1.6.1: суммы хранятся в `amount_minor` типа `BigInteger` как целое число минимальных единиц валюты, целочисленная арифметика проходит через ORM, сервисный слой и Pydantic-схемы без конвертации. `Split engine` вынесен на клиент (`frontend/src/shared/lib/money.ts`, §2.2); сервер принимает готовый список `splits` и выполняет единственную финансовую проверку — инвариант нормировки (формула (1.4)): сумма `splits[i].amount_minor` должна равняться `amount_minor` расхода.

Попарные балансы вычисляет `_calculate_balances_from_records` (ledger-based алгоритм §1.6.1); упрощение взаиморасчетов — `_simplify_balances_from_records` (жадный алгоритм §1.6.2): для каждой валюты на каждой итерации обнуляется хотя бы один участник, поэтому формируется список не более $N - 1$ переводов на валюту. Граница — верхняя оценка, а не минимум: точная минимизация числа транзакций NP-трудна [7], и жадная эвристика в отдельных конфигурациях дает число переводов выше оптимального.

Создание расхода и проведение погашения защищены идемпотентностью §1.5: клиент передает Idempotency-Key, сервис сохраняет запись в `idempotency_keys` с составным ключом (`idempotency_key`, `operation_scope`, `actor_user_id`), хэшем тела `request_hash`, кодом и телом ответа. Повтор с тем же ключом возвращает сохраненный ответ (200 OK вместо 201 Created); совпадение ключа при расхождении тела — 409 Conflict [32].

Доступ к финансовым операциям ограничен проверкой членства: перед созданием расхода или погашения сервис прямым SELECT к `groups.memberships` (`groups_reader`) проверяет, что актер — активный участник.

2.1.6 Публикация событий и уведомления

Expenses-worker доставляет события из `outbox` в брокер, замыкая событийную архитектуру §1.4.3.

Notify-service потребляет события и формирует уведомления. Идемпотентность обработки обеспечивает таблица `processed_events` с составным ключом (`event_id`, `consumer_name`): повторная доставка подтверждается без дублирующего оповещения, что дает эффективно-однократную обработку поверх at-least-once транспорта [29].

2.1.7 Работа с PostgreSQL: ORM, миграции и контроль схемы

Доступ к данным во всех сервисах построен на SQLAlchemy 2.0.

Изоляция данных закреплена отдельными ролями СУБД.

Эволюция схемы выполняется миграциями Alembic с отдельной цепочкой ревизий на каждый сервис. Применение отделено от запуска: контейнеры не выполняют alembic upgrade при старте, миграции накатываются явными целями Makefile (auth-migrate, groups-migrate, expenses-migrate, notify-migrate; генерация RSA-ключей — auth-keygen). Так рассогласование схемы превращается в явный отказ готовности — механизм, используемый и в стратегии запуска (§2.3).

Серверная часть реализует архитектуру, схему данных и финансовую модель первого раздела на уровне работающего кода: трехслойная организация шести сервисов с общим модулем common; аутентификация JWT RS256 с ротацией refresh-токенов в HttpOnly-cookie и единой точкой проверки токена в api-gateway; финансовое ядро expenses-service с серверной проверкой инварианта долей, ledger-based расчетом балансов и жадным упрощением до не более $N - 1$ переводов на валюту; идемпотентность мутаций и транзакционная запись событий в outbox с асинхронной доставкой через RabbitMQ; изоляция данных и контроль схемы на уровне СУБД через отдельные роли, миграции Alembic и schema-guard. Часть проектных решений отнесена к направлениям развития раздела 3.

2.2 Реализация клиентской части приложения

Подраздел описывает клиентское приложение, опирающееся на контракты REST API §2.1: одностраничное приложение (SPA) на React 19, собираемое сборщиком Vite.

2.2.1 Структура SPA, маршрутизация и сборка

Клиентская часть реализована как SPA без серверного рендеринга (решение зафиксировано в ADR-005, согласуется с выбором стека §1.3). Зависимости (frontend/package.json): React 19, Vite 7, React Router 7, TanStack Query v5, Zustand v5, Tailwind CSS v4 через @tailwindcss/vite.

Исходный код организован по принципу feature-sliced в четыре слоя. app/ содержит точку входа (main.tsx), провайдеры, начальную загрузку сессии (AuthBootstrap.tsx), охранник приватных маршрутов (ProtectedRoute.tsx) и границу ошибок (RouteErrorBoundary.tsx). pages/ — тонкие страницы-обертки. features/ группирует функциональность по доменам (auth, groups, expenses, notify, profile, landing), каждый делит код на api/ (хуки запросов и мутаций), ui/ (экраны) и lib/. shared/ собирает переиспользуемый код: HTTP-клиент (shared/api/client.ts), денежные утилиты (shared/lib/money.ts) и конфигурацию окружения. Деление повторяет на клиенте разделение ответственности backend-сервисов §2.1.1.

Маршрутизация построена на React Router 7 в data-режиме: дерево объявлено createBrowserRouter в routes.tsx, подключается RouterProvider. Приватные страницы загружаются по требованию через lazy() и динамический import(); каждый ленивый маршрут обернут lazyRoute, помещающей элемент в Suspense и RouteErrorBoundary, которая при сбое загрузки чанка показывает экран обновления вместо обнуления приложения.

Сборка выполняется Vite 7 (vite.config.ts) [37] с плагином React, Tailwind CSS v4 и vite-plugin-pwa. PWA добавлен аддитивным слоем: режим injectManifest с собственным service worker (src/sw.ts) и стратегией обновления prompt, при которой новая версия не активируется автоматически (компонент PwaUpdater показывает действие «Обновить»); в режиме разработки service worker отключен.

Состав ключевых feature-модулей клиентской части сведён в таблице 4.

Таблица 4 — Состав ключевых feature-модулей клиентской части

Модуль	api/ (хуки данных)	ui/ (экраны)	Назначение
auth	хуки аутентификации; стор useAuthStore	LoginPage, RegisterPage $\Sigma=0$	вход, регистрация, восстановление сессии (NF5)
groups	useCreateGroup, useCreateInvitation	GroupsListPage, CreateGroupPage, InvitePage, GroupDetailPage $N-1$	создание групп, приглашения, управление составом (F1, F2)
expenses	useSimplifiedBalances, useCreateSettlement, useGroupHistory	AddExpensePage (ExpenseForm), BalanceTab, SettlePage, HistoryTab	траты, балансы, взаиморасчёты, погашения, история (F3–F8)
notify	хуки уведомлений	лента входящих (in-app) уведомлений	оповещение участников о событиях группы (NF6)
profile	хуки профиля	экран профиля пользователя	данные учётной записи пользователя
landing	—	публичная посадочная страница	точка входа для неавторизованных посетителей

Каждый feature-модуль изолирует доменную логику по слоям: api/ инкапсулирует обращения к REST через единый клиент shared/api/client.ts и хуки TanStack Query, ui/ потребляет их данные и состояние, а каркас app/ и общий слой shared/ (денежные утилиты, конфигурация окружения) переиспользуются всеми модулями.

2.2.2 Управление состоянием: серверное и клиентское

Клиентское — идентичность пользователя и токен доступа — хранит Zustand v5 [38].

Клиентский стор (features/auth/store/useAuthStore.ts) — минимальное Zustand-хранилище с полями accessToken и user. Access-токен хранится только в памяти и никогда не записывается в localStorage, что продолжает на клиенте модель безопасности §1.4.2 и §2.1.2: refresh-токен в HttpOnly-cookie недоступен JavaScript, короткоживущий access-токен находится в памяти вкладки.

Серверное состояние сосредоточено в хуках features/*/api/. Клиент TanStack Query (app/providers.tsx) настроен на устаревание данных 30 секунд,

хранение в кеше 5 минут, отключенное повторное получение при фокусе окна и отказ от повторов запросов с кодами 4xx (опирается на класс `ApiError`).

TanStack Query держит кеш данных API с инвалидацией после мутаций и точечным фоновым обновлением, Zustand держит идентичность и access-токен в памяти без персистентного хранения. Это разделение продолжает на клиенте модель безопасности §1.4.2 и определяет логику восстановления сессии и обновления токена.

2.2.3 Взаимодействие с API: единый клиент, авторизация и обработка ошибок

Все обращения к backend проходят через единый типизированный клиент `shared/api/client.ts` поверх браузерного `fetch`.

Идемпотентность мутаций перенесена в клиент: для POST, PUT, PATCH, DELETE функция `request` генерирует `Idempotency-Key` через `crypto.randomUUID()`. Ключ создается один раз на логический запрос и переиспользуется при повторной отправке после обновления токена, поэтому сервер распознает повтор как тот же запрос; серверная сторона механизма описана в §2.1.5.

Обновление токена реализовано как цикл «401 → обновление → повтор» с дедупликацией параллельных обновлений (single-flight). При 401 клиент вызывает `refreshAccessToken`, выполняющий POST `/auth/refresh` с флагом `credentials: 'include'` и без тела: `HttpOnly-cookie` прикрепляется браузером, сервер возвращает новый access-токен, который записывается в стор.

Обработчик подключает `AuthBootstrap`: при неустранимой потере сессии он очищает кеш TanStack Query, а обнуление стора заставляет `ProtectedRoute` перенаправить на `/login`. Ошибки представлены типизированной иерархией: `ApiError` несет HTTP-статус и тело,

UnauthorizedError — неустранимый 401; ruErrorMessage (shared/api/errorMessage.ts) сопоставляет строки detail с русскими формулировками, а сбой fetch распознается как TypeError и отображается как потеря соединения.

В деградированных состояниях описанные механизмы дают пользователю предсказуемый результат. Незаметное обновление access-токена удерживает текущую страницу, а неустранимая потеря сессии возвращает на экран входа. Ответы 4xx не приводят к повторам запроса: при обращении к отсутствующему ресурсу (404) или попытке зафиксировать чужой перевод (403) пользователь видит понятную русскоязычную формулировку, а не пустой экран. Сетевой сбой отображается как потеря соединения. Сбой загрузки кода маршрута перехватывает RouteErrorBoundary, показывая экран обновления приложения вместо обнуления состояния.

2.2.4 Пользовательские сценарии

Реализованные экраны покрывают полный цикл совместного учета расходов §1.1: регистрация, формирование группы, добавление трат с распределением долей, расчет балансов, упрощение взаиморасчетов и погашение долгов. Каждый экран использует хуки серверного состояния §2.2.2 и единый клиент §2.2.3.

Вход реализован экраном LoginPage (features/auth/ui/LoginPage.tsx). На рисунке 2 показан экран входа.

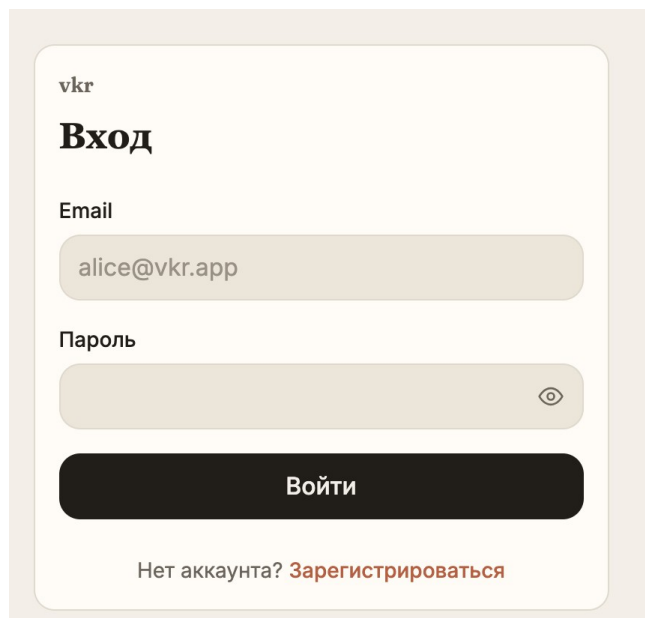


Рисунок 2 — Экран входа в приложение

Форма содержит поля email и пароля и ссылку на регистрацию.

Регистрацию реализует экран RegisterPage (features/auth/ui/RegisterPage.tsx). На рисунке 3 показан экран регистрации.

Рисунок 3 — Экран регистрации нового пользователя

Форма запрашивает имя, email, пароль и подтверждение; клиентская валидация повторяет ограничения backend (имя до 80 символов, корректный email, пароль не короче 8 символов с цифрой, совпадение подтверждения).

Создание группы реализует `CreateGroupPage` (`features/groups/ui/CreateGroupPage.tsx`) с полями названия (до 120 символов) и валюты по умолчанию; мутация `useCreateGroup` обращается к `POST /api/v1/groups`, после чего группа появляется в `GroupsListPage`.

Приглашение участника реализует `InvitePage` (`features/groups/ui/InvitePage.tsx`). Мутация `useCreateInvitation` отправляет `POST /api/v1/groups/{groupId}/invitations` ровно с одним из полей `target_user_id` либо `target_email` (XOR-инвариант схемы `InvitationCreateRequest`); при адресации по email сервер разрешает адрес в идентификатор пользователя.

Добавление расхода реализует `AddExpensePage` (`features/expenses/ui/AddExpensePage.tsx`) вокруг общей формы `ExpenseForm`, используемой и при редактировании. На рисунке 4 показана форма добавления расхода.

Добавить расход

Описание

Например, Ужин в кафе

Сумма 0.00 Валюта RUB

Кто платил

A Админ · вы T тест

Как делим

Поровну По долям

A Админ	0.00	x
T тест	0.00	x

0,00 Р / —

Отмена Сохранить

Рисунок 4 — Форма добавления расхода с распределением долей

Форма содержит описание, сумму, валюту, выбор плательщика среди активных участников и две стратегии распределения — «Поровну» и «По долям». Денежные значения обрабатываются как целые минимальные единицы (`amount_minor`) согласно §1.6: ввод преобразует `parseAmountToMinor` без `parseFloat/Number`, отображение — `Intl.NumberFormat` через `formatMinor` (`shared/lib/money.ts`).

Балансы группы показывает вкладка `BalanceTab` (`features/expenses/ui/BalanceTab.tsx`) карточки `GroupDetailPage`. На рисунке 5 показана вкладка балансов.

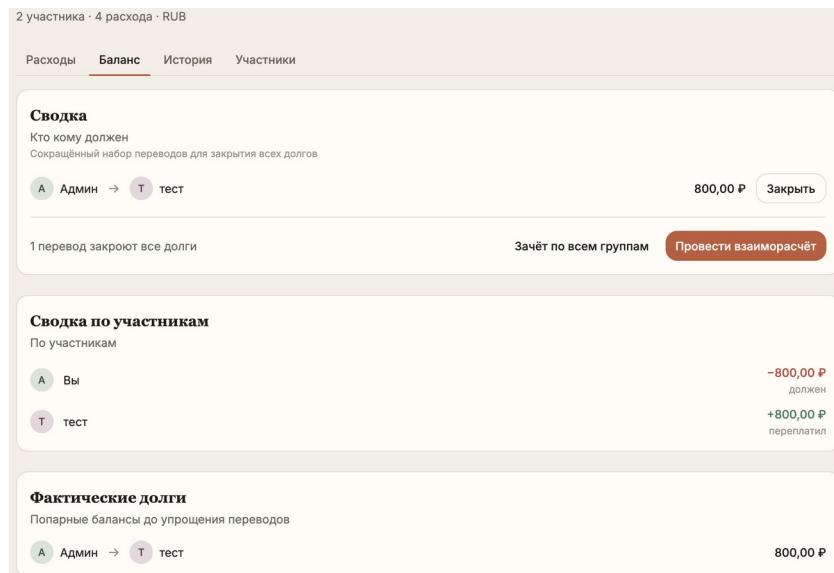


Рисунок 5 — Вкладка балансов группы со сводкой взаиморасчетов

Вкладка опирается на два эндпоинта — `GET .../{group_id}/balances` и `GET .../{group_id}/balances/simplified`. Блок «Сводка» отображает упрощенный набор переводов из `GET .../balances/simplified` (`useSimplifiedBalances`); кнопка «Заккрыть» доступна только для переводов с участием текущего пользователя, поскольку backend разрешает фиксировать лишь собственный перевод.

Погашение реализует `SettlePage` (`features/expenses/ui/SettlePage.tsx`), достижимый по кнопке «Заккрыть» или напрямую. Форма воспроизводит ограничения backend: участники различаются, текущий пользователь обязан быть стороной перевода (иначе 403), сумма не превышает зафиксированный попарный долг. Мутация `useCreateSettlement` отправляет `POST /api/v1/expenses/settlements`; при успехе балансы и история инвалидируются, выполняется возврат на вкладку балансов.

Историю показывает вкладка `HistoryTab` (`features/expenses/ui/HistoryTab.tsx`), запрашивающая `GET .../history` хуком `useGroupHistory` с постраничной подгрузкой по кнопке «Показать еще».

Реализованные экраны покрывают полный цикл совместного учета §1.1 и опираются на контракты §2.1: вход и регистрация через `/auth/*`, формирование группы и приглашение по `target_user_id` либо `email`,

добавление расхода с клиентским распределением долей и серверной проверкой инварианта, два согласованных представления балансов, погашение с проверкой участия и постраничная история.

Клиентская часть реализует SPA на React 19, потребляющее REST API серверной части §2.1 в рамках единой модели безопасности §1.4.2. Код организован по доменным слоям `app/pages/features/shared` с маршрутизацией React Router 7 в data-режиме, ленивой загрузкой приватной зоны и сборкой Vite с обязательной проверкой типов.

2.3 Реализация инфраструктуры запуска приложения

Серверная (§2.1) и клиентская (§2.2) части собираются в развертываемую на одном VPS систему слоем инфраструктуры: контейнеризацией через Docker Compose [39], единым reverse-проxy Nginx [40], терминацией TLS, отдельными ролями PostgreSQL [30] и управляемым применением миграций перед приемом трафика. Полные тексты конфигураций (Nginx, Compose-наложение, systemd-юниты, backup-скрипт) приведены в Приложении В «Конфигурации запуска приложения»; ниже излагается их архитектурная суть с опорой на файлы каталога `infra/`, `docker-compose.yml` и `docker-compose.prod.yml`.

2.3.1 Docker Compose: dev- и prod-конфигурации

Стек описан базовым `docker-compose.yml` для разработки и наложением `docker-compose.prod.yml` для production (`docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d`); наложение переопределяет только отличия боевой среды тегами слияния `!override` и `!reset` (Docker Compose $\geq$ 2.24.4). Базовый файл описывает десять сервисов — edge-проxy nginx, статический frontend, api-gateway, четыре доменных HTTP-сервиса (auth-service, groups-service, expenses-service, notify-service), фоновый expenses-worker, инфраструктурные postgres и rabbitmq — и одноразовый init-

контейнер `secrets-init`, выставляющий владельца тома `auth-secrets` под непривилегированный UID сервиса.

Сервисы разнесены по трем изолированным `docker`-сетям, что ограничивает доверие к заголовкам `X-User-*` на сетевом уровне: `edge-net` связывает `nginx`, `frontend` и `api-gateway`; `service-net` соединяет шлюз с backend-сервисами; `data-net` соединяет backend-сервисы и `worker` с `postgres` и `rabbitmq`.

Таблица 5 — Отличия dev- и production-конфигураций Docker Compose

Аспект	<code>docker-compose.yml</code> (dev)	<code>docker-compose.prod.yml</code> (prod)
Публикация портов наружу	<code>nginx</code> на <code>127.0.0.1:8080</code> ; <code>postgres</code> <code>127.0.0.1:5433</code> , <code>rabbitmq</code> <code>127.0.0.1:15672</code> , <code>frontend</code> <code>127.0.0.1:3000</code>	только <code>nginx</code> на <code>80</code> и <code>443</code> ; у <code>postgres</code> , <code>rabbitmq</code> , <code>frontend</code> — <code>ports: !reset []</code>
Политика перезапуска	не задана (краши видны разработчику)	<code>restart: unless-stopped</code> у всех сервисов
Применение миграций	вручную целями <code>Makefile</code>	отдельные <code>init</code> -контейнеры <code>*-migrate</code> через <code>service_completed_successfully</code>
Ограничения безопасности	отсутствуют	<code>no-new-privileges</code> ; <code>cap_drop: ALL</code> у backend-сервисов, <code>nginx</code> и <code>frontend</code> ; у <code>frontend</code> дополнительно <code>read_only + tmpfs /tmp</code>
Зависимости <code>nginx</code>	<code>frontend</code> и <code>api-gateway</code> (<code>service_started</code>)	<code>depends_on: !reset []</code> для <code>bootstrap</code> -режима

Главное различие сред — публикация портов. В `dev` вспомогательные порты привязаны к петлевому интерфейсу `127.0.0.1`; в `production` тег `ports: !reset` сбрасывает унаследованный список у `postgres`, `rabbitmq` и `frontend` (без явного сброса `dev`-порты протекли бы в боевую конфигурацию, так как списки `ports` при слиянии складываются), и наружу через `!override` публикуется единственный `nginx` на `80` и `443`. В `dev` этой фазы нет — миграции накатываются целями `Makefile` (§2.1.7).

Готовность проверяется проверками готовности (`readiness`) по `GET /privat/ready`: контейнер признаётся готовым только после прохождения `schema-guard` (§2.1.7), а `api-gateway` маркируется готовым лишь после

service_healthy критичных нижестоящих сервисов, что удерживает шлюз от маршрутизации к сервисам с рассогласованной схемой.

2.3.2 Конфигурация Nginx: dev, production и bootstrap

Nginx — единственная публичная точка входа: терминация TLS, маршрутизация (/api/v1/* — на api-gateway:8005, остальное — на frontend:3000) и очистка недоверенных заголовков идентичности на edge.

Очистка X-User-* — общая для dev и prod функция пограничного слоя и прямое следствие модели доверия §1.4.1: публичная точка входа позволяет любому клиенту прислать X-User-Id с чужим идентификатором.

Production-конфигурация добавляет TLS и защиту чувствительных эндпоинтов.

Защита auth-эндпоинтов от перебора реализована per-IP rate-limit'ом по ключу \$binary_remote_addr только в production: auth_login_zone — 5 запросов в минуту (/api/v1/auth/login, burst=10 nodelay), auth_refresh_zone — 30 в минуту (/api/v1/auth/refresh, burst=20), auth_register_zone — 3 в минуту (/api/v1/auth/register, burst=5).

Bootstrap-конфигурация решает проблему при первом выпуске сертификата: production-конфиг с ssl_certificate не стартует без файлов сертификата, а сертификат нельзя получить без HTTP-сервера, отдающего ACME-challenge. Переключение задается переменной NGINX_CONF в .env; для этого сценария production-наложение сбрасывает зависимости Nginx тегом depends_on: !reset.

2.3.3 TLS и сертификаты Let's Encrypt

Терминация HTTPS опирается на сертификаты Let's Encrypt, получаемые клиентом certbot по протоколу ACME http-01.

Первичный выпуск описан в deploy_vps.md: при NGINX_CONF=nginx.bootstrap.conf поднимается HTTP-only Nginx, команда

`docker compose run --rm certbot certonly --webroot --webroot-path /var/www/certbot -d $SERVER_NAME` проходит проверку `http-01` и записывает сертификат в `letsencrypt-etc`, затем `NGINX_CONF` переключается на `nginx.prod.conf` с перезапуском Nginx. Продление выполняется не внутренним циклом стека, а заданием `cron` пользователя `deploy`: ежедневный запуск в 03:00 выполняет `docker compose run --rm certbot renew` с перезагрузкой конфигурации Nginx.

2.3.4 Автозапуск и резервное копирование через `systemd`

Жизненный цикл `production`-стека и периодические операции вынесены в три `systemd`-юнита каталога `infra/systemd/`, устанавливаемых администратором в `/etc/systemd/system/` (полные тексты — в Приложении В). Резервное копирование оформлено парой «сервис плюс таймер»: `vkr-project-backup.service` запускает `infra/scripts/backup-postgres.sh`, а `vkr-project-backup.timer` инициирует его ежедневно в 03:00.

Скрипт `backup-postgres.sh` снимает дампы посхемно: для каждой из четырех схем (`auth`, `groups`, `expenses`, `notify`) выполняется `pg_dump --schema <схема> --no-owner --no-privileges`, поток сжимается `gzip` в файл `<схема>_<timestamp>.sql.gz` разбиение соответствует отдельной модели ролей и схем §1.5. Дампы остаются на том же VPS и диске, что и база, поэтому защищают в первую очередь от логических ошибок, а не от физической утраты хоста; вынос копий во внешнее объектное хранилище отнесен к направлениям развития раздела 3.

2.4 Тестирование и оценка соответствия требованиям

Реализованные в §2.1–§2.3 серверная, клиентская и инфраструктурная части проверяются на корректность с сопоставлением результата с требованиями F1–F8 и NF1–NF6 (§1.1). Центр тяжести смещен к финансовому ядру: функции `_calculate_balances_from_records()` и `_simplify_balances_from_records()`, формализованные инвариантами формул

(1.4) и (1.8) в §1.6, образуют единственный содержательно вычислительный слой и требуют наиболее строгой верификации.

2.4.1 Стратегия тестирования

Тестовое покрытие организовано по трем уровням. Нижний — модульные тесты чистых функций финансового ядра: они выполняются без базы и HTTP-слоя, поскольку движок расчета балансов реализован как детерминированная функция над списками доменных записей.

Режим хранения задается переменными `{AUTH,GROUPS,EXPENSES,NOTIFY}_STORAGE_MODE`: по умолчанию PostgreSQL, для легких модульных тестов предусмотрено хранилище в памяти (режим `memory`). Файл `backend/tests/conftest.py` задает единый источник значений окружения, генерирует эфемерную пару RSA-ключей для `auth-service` (без нее выпуск токенов завершается отказом `fail-closed`) и предоставляет фикстуры межсхемных тестов.

Клиент покрывается компонентными и модульными тестами на `vitest`: рендеринг и поведение React-компонентов, утилиты денежного форматирования и распределения долей в `shared/lib/money.ts`.

Ключевые компонентные сценарии (`vitest`, модули `*.test.tsx` и `money.test.ts`) охватывают денежные утилиты `shared/lib/money.ts` — равное распределение `distributeEqualMinor` и доначисление остатка `splitRemainderMinor` с инвариантом суммы долей, форматирование сумм в локали `ru-RU` функцией `formatMinor` — и поведение ключевых экранов: рендеринг и валидацию форм входа, регистрации и добавления расхода, переключение стратегий «Поровну» и «По долям», отображение вкладки балансов с доступностью действия «Закрыть» только для переводов с участием текущего пользователя.

Тестирование выстроено по трем уровням — модульные тесты финансового ядра, интеграционные тесты сервисного слоя против реальной

PostgreSQL и дымовая проверка готовности стека; клиент покрыт компонентными тестами vitest; сквозной браузерный e2e отсутствует и отнесен к направлениям развития.

2.4.2 Тестирование финансовых инвариантов

Финансовое ядро проверяется сочетанием примеров-эталонов и property-based тестов на основе библиотеки hypothesis [41].

Движок расчета попарных балансов проверяется модулем `test_balance_engine_properties.py`. Целочисленная арифметика в минимальных денежных единицах (§1.6) подтверждается отдельным тестом на больших значениях в пределах диапазона bigint PostgreSQL.

Распределение долей (split engine) проверяется на уровне движка и клиента.

Жадная минимизация взаиморасчетов проверяется модулем `test_expenses_simplify.py` для GET `/group_id/balances/simplified` по свойствам: сохранение нетто-сальдо (позиция участника, восстановленная из упрощенного набора переводов, совпадает с позицией классического движка, инвариант формулы (1.8)); верхняя граница не более $N - 1$ переводов на валюту, где N — число участников с ненулевым сальдо; положительность сумм и отсутствие переводов «самому себе»; идемпотентность упрощения. Граница — верхний предел, а не гарантия минимума: точная минимизация NP-трудна [7], и жадная стратегия в отдельных конфигурациях дает число переводов больше оптимального (§1.6).

Согласованность инвариантов движка со схемой данных обеспечивает интеграционный уровень. Модуль `test_expenses_check_constraints_db.py` подтверждает закрепление денежных инвариантов CHECK-ограничениями PostgreSQL: положительность `amount_minor`, соответствие `currency_code` формату ISO 4217 (`^[A-Z]{3}$`), запрет погашения «самому себе» (`from_user_id <> to_user_id`).

Денежная модель верифицирована сочетанием эталонных и property-based тестов hypothesis (для движка балансов — сохранение средств, коммутативность, нормализация встречных долгов; для упрощения — сохранение нетто-сальдо, граница не более $N - 1$ переводов на валюту, идемпотентность) и дополнительно закреплена CHECK-ограничениями PostgreSQL и интеграционными тестами межсхемной согласованности.

2.4.3 Тестирование аутентификации и безопасности

Подсистема аутентификации относится к зоне высокого риска. Ротация refresh-токенов по семействам (§1.4.2) и поведение при компрометации проверяются модулями `test_auth_security.py`, `test_auth_api_db.py`, `test_auth_edge_cases.py`: уникальность jti у токенов пользователя; константное по времени поведение входа для несуществующего пользователя (проверка пароля выполняется и при отсутствии учетной записи, снижая риск перечисления по времени ответа); корректный отказ при обновлении отозванного токена.

Параметры refresh-cookie проверяются на уровне ответа и конфигурации: `test_auth_security.py` подтверждает выставление Secure и согласованную очистку cookie при выходе (в том числе при сбое отзыва токена в базе); `test_auth_settings_samesite_validator.py` — что SameSite=Strict и SameSite=Lax принимаются, а SameSite=None отклоняется на этапе конфигурации, поскольку при отсутствии отдельного CSRF-механизма открывало бы вектор межсайтовой подделки запроса. Проверка дополняется проверкой по заголовкам Origin и Sec-Fetch-Site для cookie-зависимых эндпоинтов шлюза (`test_gateway_origin_guard.py`).

Разграничение зон доверия проверяется тестами `api-gateway`. Модуль `test_gateway_auth.py` контролирует валидацию JWT RS256 [26] по публичному ключу с кешированием JWKS и формирование X-User-* только после успешной проверки токена.

Идемпотентность мутирующих операций граничит с финансовой корректностью: повторная доставка запроса создания расхода или погашения не должна приводить к двойному начислению (§1.5). Идемпотентность потребителя событий в notify-service проверяется test_notify_consumer.py: необработанное исключение в колбэке не оставляет сообщение неподтвержденным и не разрывает канал, а повторная доставка одного события не порождает дублирующего уведомления за счет таблицы processed_events.

Подсистема аутентификации покрыта тестами ротации refresh-токенов по семействам, детекции повторного использования и параметров cookie (HttpOnly/Secure/SameSite с запретом SameSite=None); шлюз — тестами валидации JWT RS256 и формирования доверенных X-User-*; идемпотентность мутаций и потребителя событий подтверждена на уровне реальной базы и таблиц idempotency_keys и processed_events.

2.4.4 Контроль качества и автоматизация

Качество кода контролируется единым набором обязательных проверок качества (quality gate).

Проверки автоматизированы в CI. Workflow frontend-ci.yml поднимает Node 22 и pnpm, выполняет форматирование, линтинг, проверку типов, компонентные тесты и сборку с публикацией артефакта dist; он временно сокращен в связи с переписыванием клиента на React SPA — шаги stylelint, Lighthouse CI и Playwright e2e исключены до настройки инструментов, что согласуется с отсутствием e2e в §2.4.1.

Количественной метрики покрытия не зафиксировано: порог --cov и отчет в pytest и CI не настроены, поэтому покрытие характеризуется качественно.

Результаты тестирования по подсистемам сведены в таблице 6; пораздельная матрица тест-кейсов вынесена в Приложение Г.

Таблица 6 — Сводные результаты тестирования по подсистемам

Подсистема	Уровень и инструмент	Ключевые проверяемые свойства	Опорные тестовые модули
Движок балансов	unit + property-based (hypothesis)	сохранение средств $\Sigma=0$, коммутативность, нормализация встречных долгов, детерминированная сортировка	test_balance_engine_properties.py
Упрощение взаиморасчетов	unit + property-based (hypothesis)	сохранение нетто-сальдо (1.8), не более $N - 1$ переводов на валюту, идемпотентность, положительность сумм	test_expenses_simplify.py
Распределение долей	unit (pytest + vitest)	инвариант суммы долей (1.4), остаток плательщику, целочисленная арифметика	test_balance_engine_properties.py, money.test.ts
Денежные инварианты БД	integration (реальная PostgreSQL)	CHECK на amount_minor > 0, ISO 4217 валюты, запрет самопогашения	test_expenses_check_constraints_db.py
Аутентификация	unit + integration (реальная PostgreSQL)	ротация семейств токенов, детекция повторного использования, уникальность jti, параметры cookie	test_auth_security.py, test_auth_api_db.py, test_auth_settings_same_site_validator.py
API Gateway	integration (FastAPI TestClient)	валидация JWT RS256, формирование доверенных X-User-*, проверка по Origin/Sec-Fetch-Site	test_gateway_auth.py, test_gateway_origin_guard.py
Идемпотентность мутаций	integration (реальная PostgreSQL)	повтор по Idempotency-Key, срок жизни ключа, регистр валюты	test_expenses_idempotency_replay_db.py, test_expenses_idempotency_ttl.py, test_expenses_idempotency_currency_case.py
Доставка событий	integration + unit	устойчивость потребителя, идемпотентность через processed_events, контракт outbox	test_notify_consumer.py, test_outbox_to_notify_contract.py
Клиент (SPA)	компонентные (vitest)	рендеринг и поведение компонентов, денежное форматирование	модули *.test.tsx, money.test.ts

Из таблицы 6 следует, что критичные для финансовой корректности подсистемы (движки балансов и упрощения, распределение долей, денежные инварианты) покрыты на двух уровнях одновременно — property-based

проверкой чистой арифметики и интеграционными тестами против ограничений реальной базы, а зоны высокого риска (аутентификация, шлюз, идемпотентность, доставка событий) — интеграционными тестами, воспроизводящими боевую модель доступа.

Сводное соответствие реализации функциональным и нефункциональным требованиям §1.1 представлено в таблице 7.

Таблица 7 — Соответствие реализации функциональным и нефункциональным требованиям

Требование	Описание	Статус	Опора
F1	Регистрация и авторизация пользователей	реализовано	auth-service, JWT RS256, Argon2id (§2.1.2); test_auth_*.py
F2	Создание групп и управление составом	реализовано	groups-service, /invitations (§2.1.4); test_groups.py
F3	Добавление, редактирование и удаление трат	реализовано	expenses-service, /expenses (§2.1.5); test_expenses_crud.py
F4	Два режима распределения долей (равное и явные)	реализовано	split engine, валидация инварианта (1.4) (§1.5.7, §2.1.5); test_expenses_splits.py
F5	Автоматический расчет балансов	реализовано	ledger-алгоритм (§1.6.1); test_expenses_balances.py
F6	Формирование взаиморасчетов	реализовано	жадная минимизация и попарное представление (§1.6.2);

			test_expenses_simplify.py
F7	Регистрация платежей в счет погашения задолженности	реализовано	/settlements (§2.1.5); test_expenses_settlements.py
F8	Хранение и отображение истории операций	реализовано	activity_log, /history (§1.5.2, §2.1.4); test_groups_history.py
NF1	Финансовая корректность (инварианты (1.1)–(1.3) и (1.8))	реализовано	CHECK PostgreSQL, Pydantic, property-based hypothesis (§2.4.2)
NF2	Веб-интерфейс для настольных и мобильных устройств без нативного приложения	частично	адаптивная SPA-верстка (§2.2); мобильный клиент перенесен в §3.4
NF3	Локализация на русский язык и адаптация под российскую аудиторию	реализовано	русскоязычный интерфейс SPA, денежное форматирование в локали ru-RU (§2.2.4); money.test.ts
NF4	Воспроизводимость запуска и масштабируемость архитектуры	частично	Docker Compose, миграции Alembic, systemd, backup (§2.3); транзакционный Outbox с at-least-once и идемпотентным

			потребителем (§1.4.3); горизонтальное масштабирование stateless- сервисов и CDC-канал — §3.4
NF5	Защищенност ь персональных и финансовых данных	реализ овано	Argon2id, rate-limit /auth/login, HttpOnly + SameSite=Strict, JWKS (§1.4.2, §2.1.2)
NF6	Полнота журналирования операций для аудита и восстановления состояния группы	реализ овано	activity_log критичных действий (§1.5.2, §2.1.5), журнал доставки notification_deliveries; структурированное логирование и schema- guard (§2.1.7, §2.3)

Из таблицы 7 следует, что все восемь функциональных требований закрыты полностью; из шести нефункциональных требований NF1, NF3, NF5 и NF6 реализованы полностью, а NF2 и NF4 — частично, с обоснованным переносом остаточного объема в направления развития (§3.4).

Выводы по разделу 2

Раздел 2 реализовал спроектированную в разделе 1 систему совместного учета расходов на уровне работающего кода, проверенного тестами, и подтвердил ее соответствие требованиям §1.1.

Серверная часть (§2.1) реализует доменную логику в сервисном слое четырех изолированных по схемам PostgreSQL сервисов: auth-service с ротацией refresh-токенов и JWT RS256, groups-service с пригласительной

моделью и аудитом, expenses-service с движком расчета балансов, жадным упрощением до перевода и идемпотентными мутациями, notify-service с идемпотентным потребителем событий; согласованность данных при сбоях обеспечена паттерном outbox, а готовность сервиса связана с применением миграций механизмом schema-guard. Клиентская часть (§2.2) реализована как React 19 SPA с разделением серверного и клиентского состояния и денежными утилитами на целочисленной арифметике, воспроизводящими серверное правило распределения остатка.

Финансовые инварианты §1.6 верифицированы property-based тестированием (сохранение средств и коммутативность движка балансов, сохранение нетто-сальдо и граница не более $N - 1$ переводов для упрощения, инвариант суммы долей с остатком плательщику) и дополнены CHECK-ограничениями PostgreSQL и интеграционными тестами межсхемной согласованности. Сопоставление с требованиями (таблица 7) показало, что все F1–F8 и нефункциональные NF1, NF3, NF5, NF6 реализованы, а NF2 и NF4 — частично, с выносом адаптации под мобильные устройства и горизонтального масштабирования в направления развития.

Достигнутое состояние работоспособности и зафиксированные границы реализации образуют основу для раздела 3, где оцениваются архитектурные компромиссы, выполняется сопоставление с альтернативными подходами, оценка эффективности и качества решения, а частичные статусы NF2 и NF4 разворачиваются в обоснованные направления развития.

3 АНАЛИЗ РЕЗУЛЬТАТОВ РАЗРАБОТКИ И НАПРАВЛЕНИЯ РАЗВИТИЯ СИСТЕМЫ

3.1 Анализ результатов реализации архитектурных решений

В первом и втором разделах решены первые четыре задачи введения: проанализирована предметная область и сформулированы требования F1–F8 и NF1–NF6, выполнен сравнительный анализ аналогов, обоснованы архитектурный подход и технологический стек, спроектированы архитектура, схема базы данных и API, реализована финансовая модель, построены серверная и клиентская части, организована инфраструктура развертывания и проведены тестирование (таблица 6) и оценка соответствия требованиям (таблица 7, §2.4). Раздел 3 переходит от изложения сделанного к оценке: §3.1 анализирует цену закрытия архитектурных вызовов, §3.2 сопоставляет принятые решения с альтернативами того же класса, §3.3 оценивает техническую, процессную и социальную эффективность, §3.4 описывает направления развития.

Финансовое приложение в исследуемой постановке существует под двумя встречными ограничениями: финансовая корректность не допускает послаблений, тогда как инфраструктурный и временной бюджет вынуждает отказаться от части средств промышленной системы.

Первая задача — согласованность между финансовым состоянием и производным потоком уведомлений при независимых отказах компонентов. Связка `transactional outbox`, доставки «не менее одного раза» и идемпотентного потребителя (§2.1.5–2.1.6) закрывает его; оценке подлежит характер достигнутой гарантии.

Сочетание `at-least-once`-транспорта с идемпотентным потребителем не образует семантики «ровно один раз» (`exactly-once`) на уровне транспорта — такая гарантия в распределенной системе с независимыми отказами в общем случае недостижима [27]. Схема соответствует событийно-

ориентированному подходу для микросервисов [29] и закрывает надежность часть NF4.

Развязка дает и изоляцию отказов: недоступность notify-service не блокирует создание расходов, а события копятся в outbox до восстановления потребителя.

Второй вызов — разграничение зон доверия в системе из нескольких сервисов, где никто не должен судить об идентичности пользователя по непроверенным данным.

Существенно решение не кодировать в JWT членство и роли в группах: api-gateway проверяет идентичность, а права на конкретный ресурс проверяет целевой сервис по актуальному состоянию базы данных. Разграничение зон доверия закрывает NF5 и подтверждено тестами api-gateway (§2.4.3).

Третий вызов вытекает из финансовой природы системы: повторная доставка запроса на создание расхода или погашения не должна давать двойное начисление. Схема (§2.1.5) опирается на таблицу idempotency_keys с составным ключом (idempotency_key, operation_scope, actor_user_id) и соответствует рекомендациям по заголовку Idempotency-Key [32].

Идемпотентность проведена сквозь весь путь запроса, а не локализована в одном слое.

Четвертый вызов вычислительный: деление денежной суммы между участниками в общем случае не дает целого результата, а денежная единица неделима. Решение удерживает инвариант равенства суммы долей сумме расхода целочисленной арифметикой (amount_minor, тип BigInteger) и явным правилом отнесения неделимого остатка на долю плательщика (§2.1.5, §2.2).

Корректность обеспечена контуром проверки на нескольких уровнях: клиент формирует доли, expenses-service независимо проверяет нормировку и отклоняет несогласованный набор кодом 400, CHECK-ограничения PostgreSQL закрепляют положительность сумм и долей. Корректность

правила распределения остатка для широкого класса входов подтверждена property-based-тестами hypothesis (§2.4.2).

Полнота оценки требует разбора решений, в которых ради ограничений данной версии сознательно принято отступление от теоретически предпочтительного варианта. Каждый из четырех компромиссов — взвешенный выбор с понятной ценой.

Отказ от таблицы `balance_snapshot` означает вычисление балансов по запросу (`read-through`): каждое чтение агрегирует доли и погашения из списка обязательств, кэширующая `read-model` отсутствует. Намеченная в §1.4.4 стратегия `dirty + read-through` с фоновым обновлением снапшота не реализована.

Cross-schema-чтение `groups.memberships` из `expenses-service` — осознанное исключение из изоляции сервисов по схемам PostgreSQL. Членство проверяется прямым `SELECT` к чужой схеме, а не синхронным HTTP-вызовом к `groups-service`; привилегия ограничена чтением для роли `expenses_user`.

Отсюда вытекает третий, наиболее тонкий компромисс — остаточная гонка конкурентного удаления участника при `READ COMMITTED` (BK-012). Регрессионный тест покрывает детерминированный случай «участник удален до запроса», а сама гонка зафиксирована в тесте межсхемной согласованности и в `backlog` (§2.4.2).

Четвертый компромисс — отсутствие `zero-downtime-релизов`. Подстраховка — `schema-guard`: при старте каждый сервис сверяет фактическую схему с ORM-метаданными и при дрейфе уходит в `readiness=503`, не принимая трафик до устранения расхождения оператором (§2.1.7).

Архитектурные вызовы закрыты согласованными решениями, а достигнутые гарантии сформулированы без завышения. Четыре компромисса — отказ от `balance_snapshot`, cross-schema-чтение членства, остаточная гонка

READ COMMITTED, отсутствие zero-downtime-релизов — проанализированы как осознанные решения с понятной ценой, оправданной ограничениями данной версии на одном VPS, и ни один не нарушает финансовых инвариантов.

3.2 Сопоставление реализованной системы с альтернативными архитектурными подходами

Компромиссы §3.1 защитимы лишь при условии, что предложенная теорией альтернатива при целевой нагрузке снимала бы меньше проблем, чем порождала. Соответствие подходов узлам системы и статусы сведены в таблице 8.

Реализованная система хранит текущее состояние в нормализованных таблицах (`expense_splits`, `settlements`), а доменные события пишутся в `outbox` в той же транзакции и служат лишь для надежной асинхронной доставки уведомлений (§2.1.5–2.1.6); балансы вычисляются по запросу (`read-through`) при каждом запросе. Альтернативой реализованной модели является Event Sourcing: вместо текущего состояния первичен журнал доменных событий, по которому проекциями восстанавливаются состояния. В паре с CQRS журнал разделяет модель записи и оптимизированные под запросы модели чтения [27].

Полный журнал доменных событий образует естественный аудит истории операций: каждое добавление расхода, изменение долей и завершённый расчёт сохраняются как отдельные неизменяемые записи, что упрощает разбор спорных ситуаций между участниками группы. По журналу состояние группы восстанавливается на любой момент времени, поэтому ответ на вопрос «какими были балансы до конкретной операции» получается воспроизведением событий до нужной отметки, без дополнительных таблиц истории. Разделение модели записи и оптимизированных под запросы моделей чтения через CQRS позволяет масштабировать чтение балансов и лент активности независимо от записи расходов.

Издержки перехода ложатся на критичные для ядра свойства. Проекция обновляется асинхронно, поэтому чтение баланса сразу после записи расхода может вернуть устаревшее значение — *eventual consistency* на чтении становится штатной.

Транзакционная модель с Outbox дает строгую согласованность финансового состояния синхронно, в одной транзакции PostgreSQL, и надежную асинхронную доставку событий без перестройки хранения; Outbox занимает узкую нишу связки записи с брокером.

Бэкенд декомпозирован на сервисы, и собственные схемы PostgreSQL имеют *auth*, *groups*, *expenses*, *notify*.

Saga адресует проблему такой изоляции: бизнес-операцию, охватывающую несколько сервисов с собственными базами, нельзя выполнить одной ACID-транзакцией, и согласованность достигается последовательностью локальных транзакций с компенсацией при сбое на любом шаге [25]. Для проверки членства это потребовало бы заменить SELECT синхронным API-вызовом к *groups-service* либо локальной репликой членства через события (*event-carried state transfer*), а операцию над данными обоих сервисов координировать сагой.

Цена перехода для одного VPS непропорциональна выигрышу. Сага вводит промежуточные несогласованные состояния, компенсирующую логику и обработку частичных отказов — код и тесты ради сценариев, закрываемых в одном экземпляре PostgreSQL локальной транзакцией.

Cross-schema-чтение сохраняет логическое разделение ответственности (источник истины о членстве — *groups-service*, привилегия ограничена чтением) ценой одного локального SELECT. Saga и изоляция «база на сервис» — согласованное направление развития (§3.4): переход оправдан при масштабе, требующем разнести сервисы по отдельным экземплярам СУБД.

Доставка событий из outbox в RabbitMQ выполняется процессом *expenses-worker*, опрашивающим таблицу (*polling*) запросом с FOR UPDATE

SKIP LOCKED (ADR-003). Change Data Capture предлагает иной транспорт того же события: вместо опроса прикладным процессом читается журнал упреждающей записи СУБД (WAL), и изменения строк outbox извлекаются из лога репликации коннектором [27].

Операционные предпосылки CDC выходят за бюджет данной версии. Поскольку RabbitMQ зафиксирован как брокер (ADR-003 отклонил замену на Kafka), CDC означал бы либо параллельную инфраструктуру Kafka, либо отдельный коннектор со своим жизненным циклом и мониторингом — на одном VPS несоразмерное усложнение.

Polling-публикация Outbox использует имеющиеся СУБД и брокер, не вводит внешних коннекторов и реализуется одним worker-процессом, а задержка 2–5 секунд для уведомлений несущественна. CDC обоснован при интенсивности событий, на которой стоимость или задержка опроса перестают быть приемлемыми; для групп из нескольких участников этот порог не достигается.

Все четыре схемы размещены в одном экземпляре PostgreSQL, финансовые таблицы не партиционированы. При росте объема данных стандартный ответ — горизонтальное масштабирование: партиционирование разбивает таблицу на секции в пределах экземпляра, шардирование распределяет данные по нескольким экземплярам СУБД [27].

Кросс-шардовые операции теряют локальную ACID-транзакционность: если участники одной группы или связанные расход и его доли попадают на разные шарды, запись перестаёт укладываться в одну транзакцию СУБД. Денежные инварианты, прежде поддерживаемые локально, при таком распределении требуют распределённых транзакций двухфазного типа либо саг с компенсирующими операциями, что повышает сложность и стоимость записи. Запросы балансов, охватывающие участников из разных шардов, перестают выполняться одним обращением к экземпляру и требуют агрегации результатов нескольких узлов.

Единственный экземпляр PostgreSQL для целевой нагрузки далек от пределов емкости, а непартиционированная нормализованная схема проще в сопровождении и сохраняет строгую согласованность в локальных транзакциях.

В таблице 8 обобщено соответствие рассмотренных альтернативных подходов узлам реализованной системы с указанием статуса каждого выбора для данной версии приложения.

Таблица 8 — Сопоставление реализованных решений с альтернативными подходами

Реализованное решение	Альтернативный подход	Что снял бы для MVP	Цена альтернативы	Статус для MVP
Транзакционная модель + Outbox	Event Sourcing / CQRS	Темпоральные запросы, готовые read-model балансов, аудит из модели	Eventual consistency на чтении, сопровождение и версионирование проекций	Выбор оправдан; ES — при появлении темпоральной аналитики
Cross-schema-чтение memberships	Saga / распределенные транзакции, «база на сервис»	Полная изоляция данных, снятие межсхемной связанности	Компенсирующая логика, частичные отказы либо окно рассогласования реплики	Выбор оправдан; Saga — при разнесении сервисов по БД (§3.4)
Polling-публикация Outbox	Change Data Capture (чтение WAL)	Задержка публикации, нагрузка опроса при высокой интенсивности	Внешний коннектор (Debezium), слот логической репликации, рост эксплуатации	Выбор оправдан; CDC — при высокой интенсивности событий
Один экземпляр PostgreSQL	Партиционирование / шардирование по group_id	Ограничение емкости одного сервера	Ключ партиционирования в РК и ограничениях, потеря единой ACID-транзакции при шардировании	Выбор оправдан; масштабирование — при кратном росте данных

Статусы таблицы показывают регулярность принципа: для каждого узла существует более мощный подход, и в каждом случае его мощность оплачивается усложнением согласованности или эксплуатации, неоправданным при текущей нагрузке.

Сопоставление подтвердило обоснованность компромиссов §3.1 на уровне архитектурного класса решений. Принятые решения тем самым охарактеризованы как обоснованные для данной версии приложения. §3.3

переходит к измеримой эффективности результата по техническому, процессному и социальному измерениям.

3.3 Оценка эффективности и качества реализованного решения

Подразделы 3.1 и 3.2 отвечали на вопрос об обоснованности решений. Опорой служат таблица 7 (§2.4) и сводные данные тестирования, а функция подраздела — их интерпретация: что стоит за статусом «реализовано», почему два требования получили «частично», какой эффект несет каждое измерение.

Техническую эффективность определяет характер достигнутых гарантий, а не сам факт закрытия требований. Все восемь функциональных требований F1–F8 реализованы и подтверждены тестами; из шести нефункциональных четыре (NF1, NF3, NF5, NF6) реализованы полностью, два (NF2, NF4) — частично.

Центральный показатель для финансового приложения — корректность денежных расчетов, удержанная целочисленной арифметикой и многоуровневой проверкой инварианта (формула (1.4)): клиент формирует доли, сервер независимо проверяет нормировку, CHECK-ограничения PostgreSQL закрепляют положительность сумм (§3.1). Корректность правила распределения остатка и инвариант сохранения сальдо (формула (1.8)) для широкого класса входов подтверждены property-based-тестами hypothesis [41], генерирующими случайные комбинации расходов и погашений в поиске контрпримера.

Надежность доставки событий требует точной формулировки. Связка transactional outbox, at-least-once-доставки и идемпотентного потребителя (§2.1.5–2.1.6, §3.1) дает эффект однократной обработки на прикладном уровне, а не семантику exactly-once, недостижимую в распределенной системе с независимыми отказами [27].

Качество кода поддержано непересекающимися барьерами. Целочисленная денежная модель устраняет операции с плавающей запятой как источник ошибок накопления; корректность округления неделимого остатка обеспечивается явным правилом, проверяемым property-based-тестами.

Техническое измерение подтверждено полнотой закрытия F1–F8 и NF1, NF3, NF5, NF6 при честном статусе «частично» у NF2 и NF4 на периферии функциональности; корректность расчетов удержана многоуровневым контролем инвариантов и целочисленной арифметикой, надежность доставки сформулирована без завышения до exactly-once, качество кода обеспечено непротиворечивым контролем качества.

Процессная эффективность опирается на наблюдаемые следы процесса в репозитории. Результат — воспроизводимый журнал решений (журнал архитектурных решений): семь записей ADR закрепляют ключевые выборы, проектная документация и backlog фиксируют ограничения, допущения и отложенные задачи.

Property-based-тестирование сфокусировано на финансовом ядре. Тест формулирует инвариант для класса входов и поручает инструменту поиск контрпримера [41], поэтому инварианты сохранения средств, нормализации встречных долгов и границы не более $N - 1$ перевода на валюту проверяются на сотнях сгенерированных комбинаций.

Объем работы характеризуется по данным контроля версий, без перевода в денежную или нормо-часовую форму. Часть пунктов backlog — остаточная гонка удаления участника (ВК-012), полный e2e-контур (ВК-015) — внесена сознательно как зафиксированные ограничения, а не обнаружена постфактум; фиксация границ наравне с реализованным — признак управляемого процесса.

Процессное измерение выражено воспроизводимым журналом архитектурных решений spec-driven/ADR, на который опирается весь анализ

раздела 3, фокусировкой property-based-тестирования на финансовом ядре и систематическим темпом разработки, причем известные ограничения фиксировались наравне с реализованным.

Социальная эффективность вытекает из мотивации §1.2: ни один из аналогов не совмещает групповые расчеты с открытостью, русскоязычной локализацией и самостоятельным развертыванием, а данные участников везде размещены в облачной инфраструктуре поставщика. Социальный эффект измеряется по тем же критериям, по которым в §1.2 оценивались аналоги.

Самостоятельное развёртывание переносит контроль над финансовыми данными к самой группе: система собирается в воспроизводимую конфигурацию и разворачивается на произвольной инфраструктуре, закрывая сценарии с повышенными требованиями к приватности и ограниченным доступом к внешним облачным сервисам, где рыночные аналоги неприменимы. Интерфейс SPA русскоязычный, денежное форматирование использует локаль ru-RU, что закрывает NF3 и снимает языковой барьер зарубежных трекеров.

Социальное измерение определяется закрытием неудовлетворенной в §1.2 потребности: самостоятельное владение данными силами группы, русскоязычная локализация (NF3), независимость от подписочной модели и провайдера, веб-доступность без нативного приложения.

Достоверность оценки опирается на состав и тип тестов §2.4.1–§2.4.3. Структура покрытия согласована с распределением цены ошибки: чем выше риск, тем строже тип проверки.

В таблице 9 сведена оценка эффективности по трем измерениям с указанием опоры и достигнутого эффекта.

Таблица 9 — Оценка эффективности реализованного решения по измерениям

Измерение	Опора (механизм / данные)	Достигнутый эффект	Граница / оговорка
Техническая: корректность	целочисленный amount_minor, многоуровневая проверка инварианта (1.4), property-based hypothesis [41], CHECK PostgreSQL	финансовое ядро покрыто на трех уровнях, исключен класс ошибок плавающей точки	формальная корректность не доказывается, а покрывается широким классом входов
Техническая: надежность доставки	outbox + at-least-once + processed_events [27], [29]	устойчивость к потере и дублированию событий, изоляция отказов notify-service	эффект однократной обработки на прикладном уровне, не exactly-once
Техническая: качество кода	контроль качества: туру (строгие проверки), ruff, vitest, tsc; целочисленная модель	раннее обнаружение дефектов типизации и контрактов	количественный процент покрытия не измерялся
Процессная	spec-driven + ADR (журнал архитектурных решений), property-based для ядра, около 450 коммитов за ~7 недель	воспроизводимый журнал решений, систематическая разработка	трудозатраты оценены качественно, без денежной/нормо-часовой формы
Социальная	самостоятельное размещение, локаль ru-RU (NF3), отсутствие подписки, веб-доступность (NF2)	владение данными силами группы, снятие языкового и тарифного барьеров	мобильная адаптация (NF2) — частично, §3.4

Из таблицы 9 следует, что по каждому измерению эффект сопровождается явной границей: техническая корректность опирается на широкое, но не формальное покрытие; надежность доставки — прикладной эффект, а не транспортная гарантия; процессная оценка остается качественной; социальный результат ограничен частичной мобильной адаптацией.

Оценка проведена по трем измерениям без стоимостной оценки инфраструктуры и анализа развертывания. Социальная эффективность

реализует мотивацию §1.2 через самостоятельное владение данными, локализацию и независимость от подписочной модели.

3.4 Направления развития системы

Границы данной реализации, зафиксированные в §3.1–§3.3, развёрнуты ниже в четырнадцать направлений по пяти осям.

Кэширование агрегированных балансов через `balance_snapshot`.

Горизонтальное масштабирование stateless-сервисов. Частичный статус NF4 (§3.3) относится к этой границе: запуск нескольких реплик backend-сервисов не отлажен, хотя auth-, groups-, expenses-сервисы и gateway не хранят состояния, а идентичность передается в JWT и заголовках X-User-*, что делает запросы реплик взаимозаменяемыми.

Закрытие гонки конкурентного удаления участника (BK-012). Под READ COMMITTED проверка членства в expenses-service выполняется SELECT без блокировки, поэтому удаление участника между проверкой и COMMIT остается невидимым и расход может быть создан на удаленного участника без нарушения инварианта сохранения сальдо (§3.1).

Релизы blue-green и подготовка к развёртыванию без простоя (zero-downtime). Выкат версии, изменяющей схему БД, не бесшовен: миграции вынесены в явную фазу до старта сервисов (§3.1).

Мультивалютные группы с курсовой конвертацией. В данной версии приложения каждая группа ведется в одной валюте, а инвариант сохранения сальдо сформулирован внутри одной валюты (формула (1.8), §1.6).

Federated identity и внешние провайдеры аутентификации. Аутентификация выполняется auth-service по собственным учетным данным (§2.1.2), входа через внешних провайдеров нет.

Развитие api-gateway до агрегирующего BFF (Backend-for-Frontend). Gateway — единственная точка входа и проверки идентичности, поэтому он естественно подходит для агрегации сводных ответов на запросы UI [25].

Реализация — добавление агрегирующих эндпоинтов поверх прозрачного проксирования; серверный рендеринг при этом не вводится (ADR-005 отверг SSR как избыточный для приложения целиком за логином).

Остальные направления раскрыты в таблице 10 в сжатом виде из-за менее зрелого задела или отложенной активации. Завершение изоляции «база на сервис» требует замены cross-schema-чтения groups.memberships синхронным API, локальной репликой членства или Saga-координацией [25] и оправдано лишь при масштабе физического разделения баз (высокая сложность).

Таблица 10 — Направления развития системы с оценкой сложности

Группа	Направление	Ограничение MVP	Готовый задел	Сложность
Данные	balance_snapshot + read-through-кэш	Пересчет баланса на каждый запрос	Стратегия dirty + read-through намечена (§1.4.4), контракт эндпоинтов стабилен	Средняя
Данные	Изоляция «база на сервис»	Cross-schema SELECT к groups.memberships	Схемы auth, groups, expenses, notify; REST/RabbitMQ; события groups-service	Высокая
Данные	Горизонтальное масштабирование сервисов	Один экземпляр сервиса (NF4 частично)	Stateless-сервисы, идентичность в JWT/X-User-*, SKIP LOCKED у worker	Средняя
Данные	Партиционирование / шардирование	Один экземпляр PostgreSQL	Естественный ключ group_id, отсутствие связей между группами	Высокая
Надежность	Cross-service advisory-lock (BK-012)	Гонка удаления участника при READ COMMITTED	pg_advisory_xact_lock уже применен для settlement-пар	Низкая (отд. спецификация)
Надежность	Blue-green / zero-downtime релизы	Небесшовный выкат миграций схемы	Schema-guard как точка readiness-	Средняя

Группа	Направление	Ограничение MVP	Готовый задел	Сложность
			проверки	
Надежность	Автоматический replay из DLQ	Ручной разбор DLQ	Полная retry/DLQ-топология, идемпотентный потребитель	Низкая
Функциональность	Мультивалютные группы с конвертацией	Одна валюта на группу	ISO 4217 рядом с amount_minor, упрощение по валютам	Средняя / высокая (корректность)
Функциональность	Расширенная аналитика	Только балансы и история	Нормализованный список, контракт API (§1.5)	Средняя
Функциональность	Federated identity	Только собственные учетные данные	Единая точка доверия в gateway, локализация в auth-service	Средняя (высокий риск)
Frontend	Мобильная адаптация (NF2)	Нет адаптации под малые экраны	React SPA + Tailwind v4 (утилитарные классы)	Низкая
Frontend	Полный e2e-контур Playwright (BK-015)	Playwright e2e не настроен в React-стеке	Требуется добавить конфигурацию, сценарии и CI-job	Средняя
Интеграции	Агрегирующий BFF (без SSR)	Gateway не агрегирует ответы	Единая точка входа и валидации; SSR исключен ADR-005	Средняя
Интеграции	Потоковый проху (BK-016)	Буферизация тела в памяти	Прокси локализован в одной функции gateway	Низкая

Таблица 10 показывает регулярность принципа отбора: сложность определяется не объемом кода, а степенью затрагивания согласованности данных и зон высокого риска. Точечные ограничения закрываются локальными изменениями (BK-012, мобильная адаптация), тогда как изоляция данных, шардирование и e2e-контур требуют более широкой проверки архитектуры или инфраструктуры.

При росте нагрузки on-demand расчет становится узким местом: крупные группы с большой историей требуют агрегации значительного объема данных на каждый запрос. При модификации расходов снимок помечается устаревшим в той же транзакции, при обращении пересчитывается с обновлением кэша, фоновая актуализация в expenses-worker распределяет нагрузку во времени.

Две частичные границы из §2.4 получили прямое продолжение: мобильная адаптация снимает частичность NF2, горизонтальное масштабирование stateless-сервисов — частичность NF4.

Выводы по разделу 3

Третий раздел перешел от проектирования и реализации (разделы 1 и 2) к их анализу и оценке. Анализ архитектурных решений (§3.1) показал, что ключевые вызовы финансового приложения обработаны согласованными механизмами с указанием фактических границ обеспечиваемых гарантий: согласованность при частичных отказах обеспечена связкой transactional outbox, доставки at-least-once и идемпотентного потребителя, дающей эффект однократной обработки на прикладном уровне; зоны доверия сведены к единственной точке валидации JWT; корректность округления долей удержана целочисленной арифметикой.

Сопоставление с альтернативами (§3.2) подтвердило обоснованность компромиссов на уровне класса решений: для четырех узлов системы конкурирующие подходы (Event Sourcing с CQRS, Saga, CDC, шардирование) сняли бы реальное ограничение, но ценой усложнения согласованности или эксплуатации, несоразмерного нагрузке на одном VPS. Систематизация направлений развития (§3.4) развернула границы реализации в преемственный путь, привязав каждое направление к узлу системы и условиям активации.

Третий раздел не решал задачи введения заново, а проанализировал полноту и качество их решения: подтвердил соответствие результатов требованиям, охарактеризовал цену компромиссов и обозначил пространство для развития. Итоговая формулировка выводов по пяти задачам и заключение о достижении цели работы выносятся в заключение, к которому переходит изложение.

ЗАКЛЮЧЕНИЕ

В выпускной квалификационной работе исследована предметная область совместного учета расходов и разработано веб-приложение для автоматизации групповых финансовых расчетов. Результаты сопоставлены с пятью задачами, сформулированными во введении.

Анализ предметной области, сравнительный анализ существующих решений и обоснование выбора технологического стека (§1.1–§1.3) показали, что совместный учет расходов опирается на устойчивые сценарии групповых путешествий, совместного проживания и коллективных мероприятий, а денежные инварианты предметной области (сумма долей равна расходу; нетто-сальдо в пределах валюты равно нулю) формализованы и переведены в требования F1–F8 и NF1–NF6, использованные далее как критерий проектирования и проверки. Сравнение существующих коммерческих и отечественных решений выявило нишу для open-source русскоязычного приложения с возможностью самостоятельного развёртывания. Архитектурный анализ аналогов обосновал выбор стека FastAPI, PostgreSQL, RabbitMQ, React, Nginx и Docker Compose как соответствующего ограничениям одного VPS и инвариантам предметной области.

Проектирование архитектуры, схемы базы данных, программного интерфейса и формализация финансовой модели (§1.4–§1.6) задали структурный каркас системы. Спроектирована компонентная архитектура из четырех независимых сервисов — аутентификации, групп, расходов и уведомлений — с централизованной валидацией токенов JWT RS256 через api-gateway и событийной связностью поверх RabbitMQ с паттерном Transactional Outbox. Схема базы данных организована как четыре логические схемы PostgreSQL в одном экземпляре, с отдельными ролями и SELECT-грантами для кросс-схемного чтения членства; REST API спроектирован по OpenAPI 3.1 с автогенерацией контрактов из Pydantic-моделей и поддержкой идиоматичности через заголовок Idempotency-Key.

Финансовая модель формализована на двух согласованных уровнях вычислений: попарный реестр обязательств из расходов и погашений и жадный алгоритм упрощения взаиморасчетов с верхней границей $N - 1$ переводов на каждую валюту при NP-трудности точной минимизации.

Реализация серверной и клиентской частей приложения и инфраструктуры развертывания (§2.1–§2.3) перевела проектные решения в работающий код. Серверная часть собрана из Python-сервисов на FastAPI с разделением по доменным схемам PostgreSQL, ротацией refresh-токенов по семействам, идемпотентной обработкой мутаций через таблицу `idempotency_keys` и доставкой доменных событий через outbox с двухфазным claim'ом. Клиентская часть реализована как одностраничное приложение на React 19 с маршрутизацией React Router в data-режиме, ленивой загрузкой приватной зоны, разделением серверного и локального состояния (TanStack Query и Zustand) и единым API-клиентом с автоматическим обновлением access-токена. Инфраструктура развертывания собрана из контейнеров Docker Compose с reverse-proxy Nginx, TLS-терминацией Let's Encrypt, systemd-юнитами автозапуска и резервного копирования PostgreSQL; миграции применяются явной фазой перед приемом трафика и контролируются schema-guard, не допускающим дрейф фактической схемы относительно ожидаемой.

Тестирование приложения и оценка соответствия требованиям (§2.4) построены на сочетании тестирования по свойствам (*property-based testing*) финансовых инвариантов с интеграционными тестами зон высокого риска: аутентификации, шлюза, идемпотентности мутаций и доставки доменных событий. Сводное соответствие требованиям (таблица 7) фиксирует, что все восемь функциональных требований F1–F8 выполнены полностью, а из шести нефункциональных требований NF1, NF3, NF5 и NF6 реализованы полностью; NF2 (мобильная адаптация) и NF4 (распределенная надежность) реализованы частично, с обоснованным переносом остаточного объема в направления развития.

Анализ результатов реализации, сопоставление с альтернативными архитектурными подходами и определение направлений развития (§3.1–§3.4) дали интерпретацию полученной системы. Четыре ключевых архитектурных вызова — согласованность данных при частичных отказах, разграничение зон доверия, идемпотентность мутаций и корректность округления — обработаны согласованными механизмами с указанием фактических границ обеспечиваемых гарантий: семантика `at-least-once` с идемпотентным потребителем образует однократную обработку на прикладном уровне без переоценки до `exactly-once`. Четыре принятых компромисса (отказ от `balance_snapshot` в `v1`, `cross-schema`-чтение членства, остаточная гонка конкурентного удаления участника при `READ COMMITTED`, отсутствие `zero-downtime`-релизов) явно зафиксированы как граница реализации, не нарушающая финансовых инвариантов.

Сопоставление с альтернативными подходами (Event Sourcing с CQRS, Saga, Change Data Capture, шардирование PostgreSQL) показало, что для целей и бюджета данной стадии разработки принятые решения устраняют выявленные ограничения при существенно меньшей сложности (таблица 8). Эффективность реализованного решения оценена по техническому измерению (закрытие F1–F8 и NF1, NF3, NF5, NF6 при явных частичностях NF2 и NF4), процессному (воспроизводимый журнал архитектурных решений из семи записей ADR, фокусировка тестирования на финансовом ядре) и социальному (самостоятельное владение данными, русскоязычная локализация, независимость от закрытой коммерческой подписки) — таблица 9.

Направления развития системы сведены в таблицу 10 с привязкой к узлу системы, ограничению версии приложения и оценкой сложности; в первоочередные линии входят кэширование агрегированных балансов через `balance_snapshot`, нативный мобильный клиент для закрытия NF2, мультивалютные группы с курсовой конвертацией и переход к

распределенному развертыванию с горизонтальным масштабированием stateless-сервисов для снятия частичности NF4.

Сопоставление полученных результатов с целью работы — разработка веб-приложения для совместного учета расходов на основе исследования архитектурных подходов и принципов проектирования распределенных систем — подтверждает ее достижение: исследовательская часть представлена в §1.1–§1.3, разработка — в разделах 1 и 2, оценка достигнутого результата — в разделе 3. Полученный результат является завершенным, пригодным к развертыванию на одном VPS и дальнейшей эволюции в обозначенных направлениях. Научная новизна работы получена в двух элементах, заявленных во введении: формализована модель групповых взаиморасчетов с верхней границей не более $N - 1$ переводов на валюту, формируемой жадным алгоритмом упрощения, и выполнена верификация денежных инвариантов модели методом тестирования по свойствам (*property-based testing*). Практическая значимость работы состоит в том, что разработано веб-приложение, применимое для типовых сценариев совместного использования денежных средств — аренда жилья, групповые поездки, семейные и коллективные мероприятия — как open-source русскоязычная альтернатива зарубежным сервисам с возможностью самостоятельного развёртывания через Docker Compose.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Аналитический центр «Дом.РФ». Спрос на аренду жилья в России к 2030 году вырастет более чем на 25 % : совместное исследование с ВЦИОМ [Электронный ресурс] // РБК Недвижимость. – 26.02.2026. – URL: <https://realty.rbc.ru/news/69a047fa9a7947257ad4e03e> (дата обращения: 06.05.2026).
2. Аналитический центр НАФИ. В России 83 % молодежи считают семейный отдых наиболее интересным форматом путешествий : исследование совместно с программой Росмолодежи «Больше, чем путешествие» [Электронный ресурс]. – 12.02.2025. – URL: <https://nafi.ru/analytics/v-rossii-83-molodezhi-schitayut-semeynyy-otdykh-naibolee-interesnym-formatom-puteshestviy/> (дата обращения: 06.05.2026).
3. Splitwise – Split expenses with friends [Электронный ресурс]. – URL: <https://www.splitwise.com> (дата обращения: 12.04.2026).
4. Tricount – Simplify Group Expenses [Электронный ресурс] / bunq. – URL: <https://www.tricount.com> (дата обращения: 12.04.2026).
5. Егиазарян М. С. Анализ существующих приложений по ведению семейного бюджета / М. С. Егиазарян // Хроноэкономика. – 2020. – № 3 (24). – С. 110–117. – URL: <https://cyberleninka.ru/article/n/analiz-suschestvuyuschih-prilozhenii-po-vedeniyu-semeynogo-byudzheta> (дата обращения: 12.04.2026).
6. Gavrila L. A novel algorithm for clearing financial obligations between companies – an application within the Romanian Ministry of Economy / L. Gavrila, A. Popa // arXiv. – 2020. – arXiv:2012.05564. – URL: <https://arxiv.org/abs/2012.05564> (дата обращения: 12.04.2026).
7. Păţcaş C. The debts' clearing problem: a new approach / C. Păţcaş // Acta Universitatis Sapientiae, Informatica. – 2011. – Vol. 3, No. 2. – P. 192–204.

- arXiv:1111.3663. – URL: <https://arxiv.org/abs/1111.3663> (дата обращения: 12.04.2026).
- 8. Mohan M. K. Algorithm Behind Splitwise's Debt Simplification Feature [Электронный ресурс] / M. K. Mohan // Medium. – 15.12.2019. – URL: <https://medium.com/@mithunmk93/algorithm-behind-splitwises-debt-simplification-feature-8ac485e97688> (дата обращения: 12.04.2026).
- 9. Settle Up – Split expenses with friends [Электронный ресурс] / Step Up Labs. – URL: <https://settleup.io> (дата обращения: 30.05.2026).
- 10. Splid – Split group bills [Электронный ресурс]. – URL: <https://splid.app> (дата обращения: 06.05.2026).
- 11. Дзен-мани – приложение для контроля расходов [Электронный ресурс]. – URL: <https://zenmoney.ru> (дата обращения: 12.04.2026).
- 12. CoinKeeper – Приложение для учета расходов [Электронный ресурс]. – URL: <https://about.coinkeeper.me/> (дата обращения: 12.04.2026).
- 13. Т-Банк. Как разделить общий счет в ресторане : функция «Разделить» в мобильном приложении [Электронный ресурс] // Блог Т-Банка. – URL: <https://www.tbank.ru/finance/blog/split-bills/> (дата обращения: 30.05.2026).
- 14. Никитов А. Совместный сбор денег от Сбербанка: как создать общий сбор [Электронный ресурс] // Рамблер/личные финансы. – 2025. – 10 марта. – URL: <https://finance.rambler.ru/karty-i-platezhi/54315479-sovmestnyy-sbor-deneg-ot-sberbanka-kak-sozdat-obschiy-sbor/> (дата обращения: 30.05.2026).
- 15. Software Engineer, Backend : Job description [Электронный ресурс] / Splitwise, Inc. – URL: https://www.splitwise.com/jobs/software_engineer_backend (дата обращения: 30.05.2026).
- 16. Splitwise Engineering Blog [Электронный ресурс] / Splitwise, Inc. – URL: <https://blog.splitwise.com/category/dev-blog/> (дата обращения: 30.05.2026).

17. Kushwaha S. genz-splitwise-splitter / S. Kushwaha // International Journal for Multidisciplinary Research (IJFMR). – 2026. – Vol. 8, Issue 3 (May–June). – Paper ID IJFMR260377470. – URL: <https://www.ijfmr.com/papers/2026/3/77470.pdf> (дата обращения: 30.05.2026).
18. Vávra D. Settle Up is betting on the new Firebase [Электронный ресурс] / D. Vávra // Step Up Labs on Medium. – 18.05.2016. – URL: <https://medium.com/step-up-labs/settle-up-is-betting-on-the-new-firebase-d8efa1d9d0a8> (дата обращения: 30.05.2026).
19. Procházka F. Our experience with Firebase Cloud Messaging [Электронный ресурс] / F. Procházka // Step Up Labs on Medium. – 24.10.2016. – URL: <https://medium.com/step-up-labs/our-experience-with-firebase-cloud-messaging-318043e667e3> (дата обращения: 30.05.2026).
20. Egregors. ZenMoney Backup : Backup your ZenMoney data using the official API [Электронный ресурс] / Egregors. – GitHub. – URL: <https://github.com/egregors/zenmoney-backup> (дата обращения: 30.05.2026).
21. Sviridov A. A Bit on Personal Finance and Python [Электронный ресурс] / A. Sviridov // Tech blog about the boring stuff. – 06.01.2026. – URL: <https://mrlokans.work/posts/a-bit-on-personal-finance-and-python/> (дата обращения: 30.05.2026).
22. Manish Kumar M. CoinKeeper : an expense tracking web application [Электронный ресурс] / M. Manish Kumar. – GitHub. – URL: <https://github.com/mk-manishkumar/coinkeeper> (дата обращения: 30.05.2026).
23. Tezuysal A. Database Design and Modeling with PostgreSQL and MySQL: Build efficient and scalable databases for modern applications using open source databases / A. Tezuysal, I. Ahmed. – Birmingham : Packt Publishing, 2024. – 222 p. – ISBN 978-1-80323-347-5.

24. Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design / R. C. Martin. – Upper Saddle River : Prentice Hall, 2017. – 432 p. – ISBN 978-0-13-449416-6.
25. Newman S. Building Microservices: Designing Fine-Grained Systems / S. Newman. – 2nd ed. – Sebastopol : O'Reilly Media, 2021. – 612 p. – ISBN 978-1-492-03402-5.
26. Jones M. JSON Web Token (JWT) : RFC 7519 [Электронный ресурс] / M. Jones, J. Bradley, N. Sakimura. – IETF, 2015. – URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 07.05.2026).
27. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems / M. Kleppmann. – Sebastopol : O'Reilly Media, 2017. – 616 p. – ISBN 978-1-4493-7332-0.
28. RabbitMQ Documentation [Электронный ресурс]. – URL: <https://www.rabbitmq.com/docs> (дата обращения: 12.04.2026).
29. Rahmatulloh A. Event-Driven Architecture to Improve Performance and Scalability in Microservices-Based Systems / A. Rahmatulloh, F. Nugraha, R. Gunawan, I. Darmawan // 2022 International Conference Advancement in Data Science, E-learning and Information Systems (ICADEIS). – 2022. – DOI: 10.1109/ICADEIS56544.2022.10037390. – URL: <https://doi.org/10.1109/ICADEIS56544.2022.10037390> (дата обращения: 12.04.2026).
30. PostgreSQL 16 Documentation [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 12.04.2026).
31. OWASP Foundation. Password Storage Cheat Sheet [Электронный ресурс] / OWASP Cheat Sheet Series. – URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата обращения: 07.05.2026).
32. Jena J. The Idempotency-Key HTTP Header Field : draft-ietf-httpapi-idempotency-key-header (истекший Internet-Draft) [Электронный ресурс] / J. Jena, S. Dalal ; Internet Engineering Task Force. – 2025. – URL:

- <https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/>
(дата обращения: 07.05.2026).
33. Alembic Documentation [Электронный ресурс]. – URL: <https://alembic.sqlalchemy.org/en/latest/> (дата обращения: 19.04.2026).
 34. SQLAlchemy Documentation [Электронный ресурс]. – URL: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 19.04.2026).
 35. OpenAPI Specification, version 3.1.1 [Электронный ресурс]. – URL: <https://spec.openapis.org/oas/v3.1.1.html> (дата обращения: 19.04.2026).
 36. FastAPI Documentation [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com> (дата обращения: 12.04.2026).
 37. Vite 7 Documentation [Электронный ресурс]. – URL: <https://v7.vite.dev/> (дата обращения: 28.05.2026).
 38. Zustand Documentation [Электронный ресурс]. – URL: <https://zustand.docs.pmnd.rs> (дата обращения: 28.05.2026).
 39. Docker Documentation [Электронный ресурс]. – URL: <https://docs.docker.com> (дата обращения: 12.04.2026).
 40. Nginx Documentation [Электронный ресурс]. – URL: <https://nginx.org/en/docs/> (дата обращения: 19.04.2026).
 41. MacIver D. R. Hypothesis: property-based testing for Python [Электронный ресурс] / D. R. MacIver. – Документация библиотеки Hypothesis. – URL: <https://hypothesis.readthedocs.io/> (дата обращения: 31.05.2026).

СИСТЕМЫ



ПРИЛОЖЕНИЕ Б — Полный каталог REST API

Все маршруты используют префикс `/api/v1/`. Защищенные эндпоинты получают идентификатор пользователя в заголовке `X-User-Id`, сформированном `api-gateway` после валидации JWT (§1.4.2). Мутирующие операции создания расхода и записи погашения поддерживают заголовок `Idempotency-Key`.

Сервис аутентификации (auth-service)

Маршруты сгруппированы под префиксом `/api/v1/auth`.

Таблица Б.1 — Эндпоинты сервиса аутентификации

Метод	Путь	Назначение
POST	<code>/api/v1/auth/register</code>	Регистрация нового пользователя
POST	<code>/api/v1/auth/login</code>	Аутентификация и выдача access-токена
POST	<code>/api/v1/auth/refresh</code>	Ротация refresh-токена
POST	<code>/api/v1/auth/logout</code>	Отзыв refresh-токена текущей сессии
GET	<code>/api/v1/auth/me</code>	Получение профиля текущего пользователя
PATCH	<code>/api/v1/auth/me</code>	Изменение профиля (отображаемое имя)
POST	<code>/api/v1/auth/me/password</code>	Смена пароля текущего пользователя
GET	<code>/api/v1/auth/public-key</code>	Получение публичного ключа RS256 в формате PEM
GET	<code>/api/v1/auth/.well-known/jwks.json</code>	JWKS-документ для проверки подписи JWT

При POST `/register` сервис принимает `email`, `display_name` (1–80 символов) и пароль, проверяет уникальность `email`, вычисляет хэш по алгоритму Argon2id [31] и создает запись пользователя; возвращается 201 Created с полями `id`, `email`, `display_name`. POST `/login` верифицирует пароль и формирует пару токенов: access-токен возвращается в теле, refresh-токен устанавливается в HTTP-only Secure Cookie с атрибутом `SameSite=Strict`. Access-токен — JWT RS256 со временем жизни 15 минут и claims по RFC 7519 [26] (`sub`, `iss`, `aud`, `iat/nbf/exp`, `jti`); `email` в claims не включается. Refresh-токен — непрозрачная строка (`ref_` + 24 случайных байта в base64) со временем жизни 30 дней; в базе хранится только его SHA-256-хэш. Ротация с

детекцией компрометации (ротация семейства токенов) описана в §1.4.2. Эндпоинты GET /public-key и GET /.well-known/jwks.json предоставляют публичный ключ для проверки подписи на стороне api-gateway.

Сервис групп (groups-service)

Все операции требуют аутентификации; идентификатор пользователя извлекается из X-User-Id.

Таблица Б.2 — Эндпоинты сервиса групп

Метод	Путь	Назначение
POST	/api/v1/groups	Создание группы
GET	/api/v1/groups	Список групп текущего пользователя
PATCH	/api/v1/groups/{group_id}	Изменение наименования и валюты группы
POST	/api/v1/groups/{group_id}/archive	Перевод группы в архивный режим (read-only)
GET	/api/v1/groups/{group_id}/members	Список участников группы
POST	/api/v1/groups/{group_id}/members	Добавление участника в группу
DELETE	/api/v1/groups/{group_id}/members/{user_id}	Исключение участника из группы
PATCH	/api/v1/groups/{group_id}/members/{user_id}/role	Изменение роли участника группы
POST	/api/v1/groups/{group_id}/leave	Выход пользователя из группы по собственной инициативе
GET	/api/v1/groups/{group_id}/invitations	Список приглашений группы
POST	/api/v1/groups/{group_id}/invitations	Создание приглашения зарегистрированному пользователю (по target_user_id)
POST	/api/v1/groups/invitations/{id}/accept	Принятие приглашения
POST	/api/v1/groups/invitations/{id}/decline	Отклонение приглашения
POST	/api/v1/groups/invitations/{id}/revoke	Отзыв ранее созданного приглашения

При POST /groups сервис в одной транзакции создает запись группы и добавляет инициатора в memberships с ролью owner. Ролевая модель образует иерархию owner > admin > member: добавление участников разрешено только ролям owner и admin, нарушение завершается 403 Forbidden. Статус участия принимает значения active, left, removed; к операциям с расходами допускаются только участники со статусом active. Приглашения адресуются зарегистрированным пользователям по target_user_id и имеют срок действия expires_at. Email-приглашения с одноразовым токеном для незарегистрированных пользователей — направление развития v2.

Таблица Б.3 — Эндпоинты сервиса расходов

Метод	Путь	Назначение
POST	/api/v1/expenses	Создание расхода
GET	/api/v1/expenses/{expense_id}	Получение расхода с детализацией долей участников
PATCH	/api/v1/expenses/{expense_id}	Редактирование расхода
DELETE	/api/v1/expenses/{expense_id}	Удаление расхода
POST	/api/v1/expenses/settlements	Запись погашения долга
GET	/api/v1/expenses/{group_id}/balances	Текущие балансы участников группы
GET	/api/v1/expenses/{group_id}/balances/simplified	Минимизированный список переводов (не более N – 1)
GET	/api/v1/expenses/{group_id}/expenses	Список расходов группы с пагинацией
GET	/api/v1/expenses/{group_id}/history	Хронологический журнал операций группы

Запрос к POST /expenses содержит идентификатор группы, плательщика, сумму, код валюты, описание и список долей (splits). После проверки валидационных инвариантов (§1.5.5) сервис в одной транзакции создает записи в expenses, expense_splits и outbox. DELETE /expenses/{expense_id} удаляет расход, каскадно исключая строки expense_splits; снимок удаленной сущности фиксируется в activity_log в той же транзакции. Все мутирующие операции порождают события в outbox атомарно с изменением доменных таблиц. POST /expenses/settlements

фиксирует передачу денег от одного участника другому и поддерживает идемпотентность. Эндпоинты `GET /{group_id}/balances` и `GET /{group_id}/balances/simplified` вычисляют попарные задолженности и упрощенный список переводов ledger-алгоритмом, описанным в §1.6, непосредственно из таблиц `expenses`, `expense_splits`, `settlements`. Списочные эндпоинты `GET /{group_id}/expenses` и `GET /{group_id}/history` применяют offset-based пагинацию с параметрами `limit`, `offset` и полем `next_offset` в ответе; курсорная пагинация по `(created_at, id)` — направление развития v2.

ПРИЛОЖЕНИЕ В — Конфигурация запуска приложения

```
1
24 services: You, 2026-04-12 - feat/backend-auth
1  nginx:
2    image: nginx:1.27-alpine
3    depends_on:
4      frontend:
5        condition: service_started
6      api-gateway:
7        condition: service_started
8
9    ports:
10     - "127.0.0.1:${NGINX_PORT:-8080}:80"
11
12    volumes:
13     - ./infra/nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
14
15    networks:
16     - edge-net
17
18 postgres:
19   image: postgres:16-alpine
20   environment:
21     POSTGRES_DB: ${POSTGRES_DB:-vkr_app}
22     POSTGRES_USER: ${POSTGRES_USER:-vkr_admin}
23     POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-vkr_admin_password}
24     AUTH_USER_PASSWORD: ${AUTH_USER_PASSWORD}
25     GROUPS_USER_PASSWORD: ${GROUPS_USER_PASSWORD}
26     EXPENSES_USER_PASSWORD: ${EXPENSES_USER_PASSWORD}
27     NOTIFY_USER_PASSWORD: ${NOTIFY_USER_PASSWORD}
28
29   ports:
30     - "127.0.0.1:5433:5432"
31
32   volumes:
33     - pg-data:/var/lib/postgresql/data
34     - ./infra/postgres/init:/docker-entrypoint-initdb.d:ro
35     - ./infra/postgres/healthcheck.sh:/usr/local/bin/postgres-healthcheck.sh:ro
36
37   healthcheck:
38     test: ["CMD", "sh", "/usr/local/bin/postgres-healthcheck.sh"]
39     interval: 10s
40     timeout: 5s
41     retries: 5
42     start_period: 30s
43
44   networks:
45     - data-net
46
47 rabbitmq:
48   image: rabbitmq:3.13-management-alpine
49   environment:
50     RABBITMQ_DEFAULT_USER: ${RABBITMQ_DEFAULT_USER:-guest}
51     RABBITMQ_DEFAULT_PASS: ${RABBITMQ_DEFAULT_PASS:-guest}
52
53   ports:
54     - "127.0.0.1:15672:15672"
55
56   volumes:
57     - rabbitmq-data:/var/lib/rabbitmq
58     - ./infra/rabbitmq/rabbitmq.conf:/etc/rabbitmq/rabbitmq.conf:ro
```

```

75 healthcheck:
76   test: ["CMD", "rabbitmq-diagnostics", "check_running"]
77   interval: 10s
78   timeout: 10s
79   retries: 5
80 networks:
81   - data-net
82
83 frontend:
84   build:
85     context: ./frontend
86     dockerfile: Dockerfile
87     target: production
88   args:
89     VITE_API_BASE_URL: /api/v1
90     VITE_APP_NAME: vkr-mist-mva
91     VITE_LOCALE: ru-RU
92     VITE_VAPID_PUBLIC_KEY: ${VITE_VAPID_PUBLIC_KEY:-}
93   ports:
94     - "127.0.0.1:${FRONTEND_PORT:-3000}:3000"
95   healthcheck:
96     test: ["CMD", "wget", "-q0-", "http://127.0.0.1:3000/"]
97     interval: 10s
98     timeout: 5s
99     retries: 5
100  networks:
101    - edge-net
102  depends_on:
103    api-gateway:
104      condition: service_healthy
105
106 secrets-init:
107   image: busybox:1.36
108   user: "0:0"
109   command: ["sh", "-c", "chown -R 10001:10001 /app/secrets && chmod 700 /app/secrets"]
110   volumes:
111     - auth-secrets:/app/secrets
112   network_mode: "none"
113   restart: "no"
114
115 auth-service:
116   build:
117     context: ./backend
118     dockerfile: Dockerfile
119   environment:
120     APP_MODULE: services.auth.app.main:app
121     PORT: "8001"
122     SERVICE_NAME: auth-service
123     AUTH_STORAGE_MODE: ${AUTH_STORAGE_MODE:-db}
124     AUTH_DB_URL: ${AUTH_DB_URL}
125     AUTH_ACCESS_TOKEN_TTL_SECONDS: ${AUTH_ACCESS_TOKEN_TTL_SECONDS:-900}
126     AUTH_REFRESH_TOKEN_TTL_SECONDS: ${AUTH_REFRESH_TOKEN_TTL_SECONDS:-2592000}
127     AUTH_REFRESH_REUSE_GRACE_SECONDS: ${AUTH_REFRESH_REUSE_GRACE_SECONDS:-5}
128     AUTH_ISSUER: ${AUTH_ISSUER:-auth-service}
129     AUTH_AUDIENCE: ${AUTH_AUDIENCE:-vkr-api}
130     AUTH_REFRESH_COOKIE_SECURE: ${AUTH_REFRESH_COOKIE_SECURE:-false}
131     UVICORN_LIMIT_CONCURRENCY: ${UVICORN_LIMIT_CONCURRENCY:-}
132     UVICORN_LIMIT_MAX_REQUESTS: ${UVICORN_LIMIT_MAX_REQUESTS:-}
133     UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN: ${UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN:-}
134   volumes:
135     - auth-secrets:/app/secrets
136   depends_on:
137     secrets-init:
138       condition: service_completed_successfully
139     postgres:
140       condition: service_healthy
141   healthcheck:
142     test: ["CMD-SHELL", "python -c \"import urllib.request; urllib.request.urlopen('http://127.0.0.1:8001/privat/ready')\""]
143     interval: 10s
144     timeout: 5s
145     retries: 5
146     start_period: 20s
147   networks:
148     - service-net
149     - data-net
150
151 api-gateway:
152   build:
153     context: ./backend
154     dockerfile: Dockerfile
155   environment:
156     APP_MODULE: services.gateway.app.main:app
157     PORT: "8005"
158     SERVICE_NAME: api-gateway
159     GATEWAY_AUTH_JWKS_URL: http://auth-service:8001/api/v1/auth/.well-known/jwks.json
160     GATEWAY_AUTH_ISSUER: ${AUTH_ISSUER:-auth-service}
161     GATEWAY_AUTH_AUDIENCE: ${AUTH_AUDIENCE:-vkr-api}
162     GATEWAY_NOTIFY_SERVICE_BASE: http://notify-service:8004
163     GATEWAY_TRUSTED_ORIGINS: ${GATEWAY_TRUSTED_ORIGINS:-http://localhost:${NGINX_PORT:-8080}}
164     UVICORN_PROXY_HEADERS: "false"
165     UVICORN_LIMIT_CONCURRENCY: ${UVICORN_LIMIT_CONCURRENCY:-}
166     UVICORN_LIMIT_MAX_REQUESTS: ${UVICORN_LIMIT_MAX_REQUESTS:-}
167     UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN: ${UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN:-}
168
169 You, 2026-04-12 - feat/backend-auth
170
171 main - 92 docker-compose.yml

```

```

20 UVICORN_LIMIT_CONCURRENCY: ${UVICORN_LIMIT_CONCURRENCY:-}
21 UVICORN_LIMIT_MAX_REQUESTS: ${UVICORN_LIMIT_MAX_REQUESTS:-}
22 UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN: ${UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN:-}
23 depends_on:
24   auth-service:
25     condition: service_healthy
26   groups-service:
27     condition: service_healthy
28   expenses-service:
29     condition: service_healthy
30   notify-service:
31     condition: service_healthy
32 healthcheck:
33   test: ["CMD-SHELL", "python -c \"import urllib.request; urllib.request.urlopen('http://127.0.0.1:8005/privat/ready')\""]
34   interval: 10s
35   timeout: 5s
36   retries: 5
37 networks:
38   - edge-net
39   - service-net
40
41 ou, 2026-04-19 - feat/auth, group, tests
42 groups-service:
43   build:
44     context: ../backend
45     dockerfile: Dockerfile
46   environment:
47     APP_MODULE: services.groups.app.main:app
48     PORT: "8002"
49     SERVICE_NAME: groups-service
50     GROUPS_STORAGE_MODE: ${GROUPS_STORAGE_MODE:-db}
51     GROUPS_DB_URL: ${GROUPS_DB_URL}
52     UVICORN_LIMIT_CONCURRENCY: ${UVICORN_LIMIT_CONCURRENCY:-}
53     UVICORN_LIMIT_MAX_REQUESTS: ${UVICORN_LIMIT_MAX_REQUESTS:-}
54     UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN: ${UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN:-}
55   depends_on:
56     postgres:
57       condition: service_healthy
58   healthcheck:
59     test: ["CMD-SHELL", "python -c \"import urllib.request; urllib.request.urlopen('http://127.0.0.1:8002/privat/ready')\""]
60     interval: 10s
61     timeout: 5s
62     retries: 5
63     start_period: 20s
64   networks:
65     - service-net
66     - data-net
67
68 expenses-service:
69   build:
70     context: ../backend
71     dockerfile: Dockerfile
72   environment:
73     APP_MODULE: services.expenses.app.main:app
74     PORT: "8003"
75     SERVICE_NAME: expenses-service
76     EXPENSES_STORAGE_MODE: ${EXPENSES_STORAGE_MODE:-db}
77     EXPENSES_GROUPS_SOURCE: ${EXPENSES_GROUPS_SOURCE:-db}
78     EXPENSES_DB_URL: ${EXPENSES_DB_URL}
79     UVICORN_LIMIT_CONCURRENCY: ${UVICORN_LIMIT_CONCURRENCY:-}
80     UVICORN_LIMIT_MAX_REQUESTS: ${UVICORN_LIMIT_MAX_REQUESTS:-}
81     UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN: ${UVICORN_TIMEOUT_GRACEFUL_SHUTDOWN:-}
82   depends_on:
83     postgres:
84       condition: service_healthy
85     rabbitmq:
86       condition: service_healthy
87   healthcheck:
88     test: ["CMD-SHELL", "python -c \"import urllib.request; urllib.request.urlopen('http://127.0.0.1:8003/privat/ready')\""]
89     interval: 10s
90     timeout: 5s
91     retries: 5
92     start_period: 20s
93   networks:
94     - service-net
95     - data-net
96
97 notify-service:
98   build:
99     context: ../backend
100    dockerfile: Dockerfile
101   environment:
102     APP_MODULE: services.notify.app.main:app
103     PORT: "8004"
104     SERVICE_NAME: notify-service
105     APP_ENV: ${APP_ENV:-development}
106     NOTIFY_STORAGE_MODE: ${NOTIFY_STORAGE_MODE:-db}
107     NOTIFY_DB_URL: ${NOTIFY_DB_URL}
108     NOTIFY_RABBITMQ_URL: ${NOTIFY_RABBITMQ_URL:-amqp://guest:guest@rabbitmq:5672/}
109     NOTIFY_CONSUMER_NAME: ${NOTIFY_CONSUMER_NAME:-notify-service.email}
110     NOTIFY_RETRY_TTL_MS: ${NOTIFY_RETRY_TTL_MS:-30000}
111     NOTIFY_MAX_RETRIES: ${NOTIFY_MAX_RETRIES:-5}
112     NOTIFY_VAPID_PRIVATE_KEY_PATH: ${NOTIFY_VAPID_PRIVATE_KEY_PATH:-/run/secrets/vapid_private.pem}
113     NOTIFY_VAPID_PUBLIC_KEY: ${NOTIFY_VAPID_PUBLIC_KEY:-}

```

```

84 NOTIFY_RABBITMQ_URL: ${NOTIFY_RABBITMQ_URL:-amqp://guest:guest@rabbitmq:5672/}
83 NOTIFY_CONSUMER_NAME: ${NOTIFY_CONSUMER_NAME:-notify-service.email}
82 NOTIFY_RETRY_TTL_MS: ${NOTIFY_RETRY_TTL_MS:-30000}
81 NOTIFY_MAX_RETRIES: ${NOTIFY_MAX_RETRIES:-5}
80 NOTIFY_VAPID_PRIVATE_KEY_PATH: ${NOTIFY_VAPID_PRIVATE_KEY_PATH:-/run/secrets/vapid_private.pem}
79 NOTIFY_VAPID_PUBLIC_KEY: ${NOTIFY_VAPID_PUBLIC_KEY:-}
78 NOTIFY_VAPID_SUBJECT: ${NOTIFY_VAPID_SUBJECT:-mailto:admin@vkr-mist-mva.local}
77 NOTIFY_WEBPUSH_TIMEOUT_SECONDS: ${NOTIFY_WEBPUSH_TIMEOUT_SECONDS:-10}
76 NOTIFY_WEBPUSH_TTL_SECONDS: ${NOTIFY_WEBPUSH_TTL_SECONDS:-86400}
75 NOTIFY_WEBPUSH_MAX_CONCURRENCY: ${NOTIFY_WEBPUSH_MAX_CONCURRENCY:-16}
74 UVICORN_LIMIT_CONCURRENCY: ${UVICORN_LIMIT_CONCURRENCY:-}
73 UVICORN_LIMIT_MAX_REQUESTS: ${UVICORN_LIMIT_MAX_REQUESTS:-}
72 UVICORN_TIMEOUT_Graceful_SHUTDOWN: ${UVICORN_TIMEOUT_Graceful_SHUTDOWN:-}
71 volumes:
70 - ./infra/secrets/notify/vapid_private.pem:/run/secrets/vapid_private.pem:ro
69 depends_on:
68 postgres:
67 | condition: service_healthy
66 rabbitmq:
65 | condition: service_healthy
64 healthcheck:
63 test: ["CMD-SHELL", "python -c \"import urllib.request; urllib.request.urlopen('http://127.0.0.1:8004/privat/ready')\""]
62 interval: 10s
61 timeout: 5s
60 retries: 5
59 start_period: 20s
58 networks:
57 - service-net
56 - data-net
55
54 expenses-worker:
53 build:
52 | context: ./backend
51 | dockerfile: Dockerfile
50 environment:
49 SERVICE_NAME: expenses-worker
48 EXPENSES_DB_URL: ${EXPENSES_DB_URL}
47 EXPENSES_RABBITMQ_URL: ${EXPENSES_RABBITMQ_URL:-amqp://guest:guest@rabbitmq:5672/}
46 EXPENSES_OUTBOX_POLL_INTERVAL_SECONDS: ${EXPENSES_OUTBOX_POLL_INTERVAL_SECONDS:-2}
45 EXPENSES_OUTBOX_BATCH_SIZE: ${EXPENSES_OUTBOX_BATCH_SIZE:-10}
44 EXPENSES_OUTBOX_MAX_PUBLISH_ATTEMPTS: ${EXPENSES_OUTBOX_MAX_PUBLISH_ATTEMPTS:-8}
43 EXPENSES_OUTBOX_MAX_BACKOFF_SECONDS: ${EXPENSES_OUTBOX_MAX_BACKOFF_SECONDS:-300}
42 EXPENSES_OUTBOX_PUBLISH_TIMEOUT_SECONDS: ${EXPENSES_OUTBOX_PUBLISH_TIMEOUT_SECONDS:-5}
41 EXPENSES_OUTBOX_PROCESSING_TIMEOUT_SECONDS: ${EXPENSES_OUTBOX_PROCESSING_TIMEOUT_SECONDS:-60}
40 command: python -m services.worker.main
39 depends_on:
38 postgres:
37 | condition: service_healthy
36 rabbitmq:
35 | condition: service_healthy
34 networks:
33 - data-net
32
31 groups-worker:
30 build:
29 | context: ./backend
28 | dockerfile: Dockerfile
27 environment:
26 SERVICE_NAME: groups-worker
25 WORKER_DB_URL: ${GROUPS_DB_URL}
24 WORKER_OUTBOX_SCHEMA: groups
23 WORKER_RABBITMQ_URL: ${EXPENSES_RABBITMQ_URL:-amqp://guest:guest@rabbitmq:5672/}
22 WORKER_OUTBOX_POLL_INTERVAL_SECONDS: ${WORKER_OUTBOX_POLL_INTERVAL_SECONDS:-2}
21 WORKER_OUTBOX_BATCH_SIZE: ${WORKER_OUTBOX_BATCH_SIZE:-10}
20 WORKER_OUTBOX_MAX_PUBLISH_ATTEMPTS: ${WORKER_OUTBOX_MAX_PUBLISH_ATTEMPTS:-8}
19 WORKER_OUTBOX_MAX_BACKOFF_SECONDS: ${WORKER_OUTBOX_MAX_BACKOFF_SECONDS:-300}
18 WORKER_OUTBOX_PUBLISH_TIMEOUT_SECONDS: ${WORKER_OUTBOX_PUBLISH_TIMEOUT_SECONDS:-5}
17 WORKER_OUTBOX_PROCESSING_TIMEOUT_SECONDS: ${WORKER_OUTBOX_PROCESSING_TIMEOUT_SECONDS:-60}
16 command: python -m services.worker.main
15 depends_on:
14 postgres:
13 | condition: service_healthy
12 rabbitmq:
11 | condition: service_healthy
10 networks:
9 - data-net
8
7 networks:
6 edge-net:
5 | driver: bridge
4 service-net:
3 | driver: bridge
2 data-net:
1 | driver: bridge

```

ПРИЛОЖЕНИЕ Г — Матрица тест-кейсов

Матрица детализирует сводную таблицу 6 и оценку соответствия требованиям (таблица 7, §2.4): для каждого требования F1–F8 и NF1–NF6 указаны опорные тестовые модули каталогов backend/tests/ и frontend/src/, уровень проверки и проверяемые свойства. Уровни: U — модульный тест (unit), P — property-based (hypothesis [41]), I — интеграционный тест против реальной PostgreSQL, C — компонентный тест клиента (vittest).

Г.1 Функциональные требования

Таблица Г.1 — Тест-кейсы функциональных требований F1–F8

Требование	Тестовый модуль	Уровень	Проверяемые свойства
F1. Регистрация и авторизация	test_auth_security.py	U+I	уникальность jti; константное по времени поведение входа для несуществующего пользователя; отказ при обновлении отозванного токена; параметры refresh-cookie (Secure, очистка при выходе)
	test_auth_api_db.py	I	детекция повторного использования refresh-токена и инвалидация семейства на реальной базе
	test_auth_edge_cases.py	I	граничные сценарии ротации и отзыва токенов
F2. Создание групп и приглашения модель	test_groups_api_db.py	I	создание группы в одной транзакции, роли и статусы членства
	test_groups_invitations.py	I	жизненный цикл приглашений (pending/accepted/declined/expired/revoked)
F3. CRUD трат с плательщиком, суммой, описанием, участниками	test_expenses_crud_api.py	I	создание, редактирование, удаление расхода; разделение paid_by_user_id и created_by_user_id
	test_expenses_api_db.py	I	согласованность записи расхода и долей в транзакции
F4. Равное деление и	test_balance_engine_properties.py	U+P	режимы equal/custom; инвариант суммы долей (1.4);

Требование	Тестовый модуль	Уровень	Проверяемые свойства
явные доли			остаток плательщику
	money.test.ts	C	distributeEqualMinor, splitRemainderMinor — клиентское правило распределения, согласованное с сервером
F5. Автоматический расчет балансов (1.2)	test_balance_engine_properties.py	U+P	_calculate_balances_from_records(); попарные позиции; обнуление совокупного баланса
F6. Взаиморасчеты (1.3): нормализация и упрощение до $N - 1$ переводов $N - 1$	test_expenses_simplify.py	U+P	_simplify_balances_from_records(); сохранение нетто-сальдо (1.8); граница не более $N - 1$ $N - 1$ перевода на валюту; идемпотентность; отсутствие переводов «самому себе»
F7. Регистрация погашений с обновлением балансов	test_expenses_settlement_isolation.py	I	фиксация погашения; изоляция операций
	test_balance_engine_properties.py	U+P	точное уменьшение задолженности на сумму адресного погашения; схлопывание встречных задолженностей
F8. История операций группы	test_expenses_history_payload.py	I	формирование и отображение истории операций

Г.2 Нефункциональные требования

Таблица Г.2 — Тест-кейсы нефункциональных требований NF1–NF6

Требование	Тестовый модуль	Уровень	Проверяемые свойства
NF1. Финансовая корректность — инварианты (1.1)–(1.3)	test_balance_engine_properties.py	U+P	сохранение средств (чистых позиций по валюте) $\Sigma = 0$; коммутативность относительно порядка операций; нормализация встречных долгов; строгая положительность сумм; целочисленная арифметика в пределах bigint
	test_expenses_simplify.py	U+P	сохранение нетто-сальдо (1.8); граница $N - 1$
	test_expenses_check_constraints_db.py	I	CHECK PostgreSQL: amount_minor > 0, currency_code ~ '^ [A-Z] {3} \$',

Требование	Тестовый модуль	Уровень	Проверяемые свойства
			from_user_id <> to_user_id
	test_expenses_cross_schema_consistency.py	I	межсхемная согласованность expenses ↔ groups: расход не создается для исключенного участника (с фиксацией границы гонки под READ COMMITTED)
	test_expenses_idempotency_replay_db.py	I	повтор по Idempotency-Key возвращает закешированный ответ без двойного начисления
	test_expenses_idempotency_ttl.py	I	срок жизни ключа идемпотентности
	test_expenses_idempotency_currency_case.py	I	нормализация регистра кода валюты до вычисления хеша
NF2. Веб-интерфейс для настольных и мобильных устройств	модули *.test.tsx	C	рендеринг и поведение React-компонентов (адаптация под мобильные — направление развития, §3.4)
NF3. Локализация на русский язык	money.test.ts	C	форматирование валют в локали ru-RU
NF4. Воспроизводимость развертывания и масштабируемость	test_outbox_to_notify_contract.py	U+I	контракт outbox → notify; событийная развязка (горизонтальное масштабирование — направление развития, §3.4)
	test_notify_consumer.py	U+I	устойчивость потребителя; идемпотентность доставки через processed_events
NF5. Защищенность персональных и финансовых данных	test_gateway_auth.py	I	валидация JWT RS256 [26] по публичному ключу с кешем JWKS; формирование X-User-* только после проверки токена; устойчивость пути аутентификации к сбою записи в аудит
	test_gateway_origin_guard.py	I	проверка по заголовкам Origin и Sec-Fetch-Site для cookie-зависимых эндпоинтов
	test_auth_settings_same_site_validator.py	U	прием SameSite=Strict/Lax, отклонение SameSite=None на этапе конфигурации
NF6. Полнота журналирования для аудита	test_expenses_audit_records_denials.py	I	фиксация отказов в audit_log; структурированные логи с request_id

Количественная метрика покрытия (`--cov`) в проекте не настроена, поэтому покрытие характеризуется качественно (§2.4.4); тестовая база включает 90 модулей `test_*.py` для `backend` и 59 файлов тестов клиента.

ПРИЛОЖЕНИЕ Д — Архитектура системы

